

Final Project

Sophia Mummert

2026-05-01

```
#-----#
#                               #
#-----#

#demography table
names(demography_table)[names(demography_table) == "alive_now?"] = "Status"
names(demography_table)[names(demography_table) == "age_categeory_first_seen"] = "Age_Cat"
names(demography_table)[names(demography_table) == "Code"] = "Focal_ID"

demog = demography_table %>%
  select(Name, Focal_ID, Sex, NV_Original, Status, Age_Cat) %>%
  filter(Focal_ID %in% c("ES", "MS", "ZN", "RB", "LK", "GR", "KB", "BV", "SU"))

demog[1,6] = "Parous"

#####----- plant foods df -----#####
food_list$Food_ID = tolower(food_list$Species)
names(food_list)[names(food_list) == "Native, Alien, or Alien Invasive?"] = "Status"
food_list$Status = tolower(food_list$Status)

food_list = food_list %>%
  distinct(Food_ID, .keep_all = TRUE)

#####----- full bintervention -----#####
bintervention$Food_ID = tolower(bintervention$Food_ID)
bintervention$Food_part = tolower(bintervention$Food_part)

bintervention = bintervention %>%
  left_join(demog %>% select(Focal_ID, Sex, Age_Cat),
    by = "Focal_ID") %>%
  left_join(food_list %>% select(Food_ID, Status),
    by = "Food_ID")

#now just going to be alien versus indigenous, so changing the "alien invasive" to just "alien"
bintervention = bintervention %>%
  mutate(Status = if_else(str_detect(Status, "alien invasive"), "alien", Status))

#making the rest of Status NAs unknown
bintervention = bintervention %>%
  mutate(Status = coalesce(Status, "unknown"))
```

```

#more precise to say "unidentified"
bintervention$Status[bintervention$Status == "unknown"] = "unidentified"

##### time cleaning #####

#cleaning time function taken from previous project
convert_mixed_time = function(x) {
  if (is.factor(x)) x = as.character(x)

  if (grepl("^1899-12-31", x)) {
    time_part = sub("1899-12-31\\s+", "", x)
    try_excel_hms = suppressWarnings(as.numeric(lubridate::hms(time_part)))
    if (!is.na(try_excel_hms)) return(try_excel_hms)
  }

  try_hms = suppressWarnings(as.numeric(lubridate::hms(x)))
  if (!is.na(try_hms)) return(try_hms)

  num = suppressWarnings(as.numeric(x))
  if (!is.na(num)) return(num * 86400)

  return(NA_real_)
}

### binter_focal bc I need to get the TMF start times ###
binter_focal$Focal_behavior = tolower(binter_focal$Focal_behavior)

binter_focal = binter_focal %>%
  select(Focal_Index, Date, ScanNumber, ScanTime, Focal_behavior) %>%
  filter(ScanNumber == 1) %>%
  filter(Focal_behavior == "feed")

binter_focal = binter_focal %>%
  mutate(
    ScanTime = apply(ScanTime, convert_mixed_time)) %>%
  mutate(ScanEnd = ScanTime + 600)

#remove duplicate groups in binter_focal that need to be cleaned
time_binter = binter_focal %>%
  group_by(Focal_Index) %>%
  summarise(
    ScanTime = first(ScanTime),
    ScanEnd = first(ScanEnd),
    .groups = "drop"
  )

#clean time columns to convert to seconds, and use TMF as start times when
#start times are missing
bintervention = bintervention %>%
  mutate(start_time_sec = apply(Start_time, convert_mixed_time),
    end_time_sec = apply(End_time, convert_mixed_time),
    TMF_start = apply(TMF_start, convert_mixed_time),

```

```

    TMF_end = supply(TMF_end, convert_mixed_time))

bintervention = bintervention %>%
  mutate(
    start_time_sec = coalesce(start_time_sec, TMF_start),
    end_time_sec   = coalesce(end_time_sec, TMF_end),
    Duration_sec   = end_time_sec - start_time_sec
  )

bintervention = bintervention %>%
  select(Focal_Index, Day, Month, Year, Date, Focal_ID,
         Food_ID, Food_part, Context_and_comments, Status, Sex,
         Age_Cat, TMF_start, start_time_sec, end_time_sec, TMF_end,
         Duration_sec)

#honestly this whole time_binter and time_bintervention fix took legitimately
#2 days to figure out and I just kept throwing things at the code until I got it
#to work work and I'm almost positive there's a better way to do it but this is
#what I got.

#basically, if the start and end times are there, keep
#next, if the TMF start or end was there (I added to my local excel file), then
#use those to fill in missing start or end time.
#next, if bout start or end is empty AND TMF start or end time is empty, pull
#from the focal behavior scan with corresponding focal index (which I input for
#those missing focal_index in feeding tab on local file). Therefore, binterfocal
#is filtered for: behavior = feed, scan = 1. SO that SHOULD account for all the
#missing TMF start, then automatically adds 600 seconds for TMF end. I just
#manually fill in the TMF ends for entries that had a start time but "in prog"
#for end. This accounts for all entries marked "in prog". The only 10 leftover
#were from TMFs that have weird discrepancies in the field datasheets or are
#missing the field sheets entirely.

time_bintervention = bintervention %>%
  left_join(time_binter, by = "Focal_Index") %>%
  mutate(
    TMF_start = ifelse(
      is.na(TMF_start) & is.na(start_time_sec),
      ScanTime,
      TMF_start
    ),
    TMF_end = ifelse(
      is.na(TMF_end) & is.na(end_time_sec),
      ScanEnd,
      TMF_end
    )
  ) %>%
  select(-ScanTime, -ScanEnd) %>%
  mutate(
    start_time_sec = coalesce(start_time_sec, TMF_start),
    end_time_sec   = coalesce(end_time_sec, TMF_end),
    Duration_sec   = end_time_sec - start_time_sec
  )

```

```

#combine into one AF cat
time_bintervention = time_bintervention %>%
  mutate(
    Sex_Age = case_when(
      Sex == "female" & Age_Cat %in% c("Parous", "Nulliparous") ~ "Adult female",
      Age_Cat == "Juvenile" ~ "Juvenile",
      TRUE ~ paste(Age_Cat, Sex)
    )
  )

#I removed the 10 entries that still have NA in the duration section
#because I cannpt "clean" them in my dataset (missing, etc)
clean_bintervention = time_bintervention %>%
  select(Focal_Index, Date, Focal_ID,
    Food_ID, Food_part, Status,
    Context_and_comments,
    Duration_sec, Sex, Age_Cat, Sex_Age) %>%
  drop_na(Duration_sec)
#####

# food part cleaning #
unique(bintervention$Food_part)
#starting with 91 unique entries

clean_food_part = clean_bintervention %>%
  select(Focal_Index, Focal_ID, Date, Food_ID, Food_part,
    Context_and_comments, Duration_sec, Sex_Age, Status) %>%
  mutate(
    Food_part_raw = str_to_lower(Food_part),
    Food_part_raw = str_trim(Food_part_raw),

    #all of this part is cleaning but in R because it seemed easier to catch things
    Food_part_raw = str_replace_all(Food_part_raw, "\\(", ""),
    Food_part_raw = str_replace_all(Food_part_raw, "\\)", ""),

    Food_part_raw = str_replace_all(Food_part_raw, "leaves - mature", "mature leaf"),
    Food_part_raw = str_replace_all(Food_part_raw, "mature and young leaves",
      "mature leaf and young leaf"),
    Food_part_raw = str_replace_all(Food_part_raw, "fruti", "fruit"),
    Food_part_raw = str_replace_all(Food_part_raw, "fruits", "fruit"),
    Food_part_raw = str_replace_all(Food_part_raw, "leafs", "leaf"),
    Food_part_raw = str_replace_all(Food_part_raw, "leaves", "leaf"),
    Food_part_raw = str_replace_all(Food_part_raw, "leave", "leaf"),
    Food_part_raw = str_replace_all(Food_part_raw, "young sapling", "young leaf"),
    Food_part_raw = str_replace_all(Food_part_raw, "leavee", "leaf"),
    Food_part_raw = str_replace_all(Food_part_raw, "shoots?|short shoot|shoots", "shoot"),
    Food_part_raw = str_replace_all(Food_part_raw, "roots?", "root"),
    Food_part_raw = str_replace_all(Food_part_raw, "seeds?|brown seeds", "seed"),
    Food_part_raw = str_replace_all(Food_part_raw, "seed pods", "seed"),
    Food_part_raw = str_replace_all(Food_part_raw, "flowers?", "flower"),
    Food_part_raw = str_replace_all(Food_part_raw, "outer ?bark|inner bark", "bark"),
    Food_part_raw = str_replace_all(Food_part_raw, "matures leaf", "mature leaf"),
    Food_part_raw = str_replace_all(Food_part_raw, "mature leafs", "mature leaf"),

```

```

Food_part_raw = str_replace_all(Food_part_raw, "immature leafs|immature leaf", "young leaf"),
Food_part_raw = str_replace_all(Food_part_raw, "leaf,young", "young leaf"),
Food_part_raw = str_replace_all(Food_part_raw, "matures leaf", "mature leaf"),
Food_part_raw = str_replace_all(Food_part_raw, "mature leafs", "mature leaf"),
Food_part_raw = str_replace_all(Food_part_raw, "immature leafs|immature leaf", "young leaf"),
Food_part_raw = str_replace_all(Food_part_raw, "leaf,young", "young leaf"),
Food_part_raw = str_replace_all(Food_part_raw, "leaf-young", "young leaf"),
Food_part_raw = str_replace_all(Food_part_raw, "leaf-mature", "mature leaf"),
Food_part_raw = str_replace_all(Food_part_raw, "mature leavee", "mature leaf"),
Food_part_raw = str_replace_all(Food_part_raw, "fruit unripe", "unripe fruit"),
#making stems = shoots for these grasses, there isn't clear standardized terminology
#in the literature...
Food_part_raw = str_replace_all(Food_part_raw, "stem base|stem", "shoot"),
Food_part_raw = str_replace_all(Food_part_raw, "dirt|soil", "soil"),
Food_part_raw = str_replace_all(Food_part_raw, "body", "insect"),
Food_part_raw = str_replace_all(Food_part_raw, "peel", "trash"),
Food_part_raw = str_replace_all(Food_part_raw, "leaves/stick", "leaves,stick"),
Food_part_raw = str_replace_all(Food_part_raw, "na|unknown|unk", "other"),

Food_part_raw = str_replace_all(Food_part_raw, "\\s*,\\s*", ","),

Food_part_raw = str_replace_all(Food_part_raw, " and |/&|-|\\.", ","),
Food_part_raw = str_replace_all(Food_part_raw, ",", "+", ","),

Food_part_raw = str_trim(Food_part_raw)) %>%

#some have more than 1 plant part, separate them to be counted as separate ?
mutate(
  n_parts = ifelse(is.na(Food_part_raw), 1,
    str_count(Food_part_raw, ",") + 1)
) %>%

separate_rows(Food_part_raw, sep = ",") %>%
mutate(Food_part_raw = str_trim(Food_part_raw)) %>%

#can't keep rewriting this cleaning code so adding breakthrough typos here
mutate(
  Food_part_raw = str_replace_all(Food_part_raw, "shoots", "shoot"),
  Food_part_raw = str_replace_all(Food_part_raw, "insects", "insect"),
  Food_part_raw = str_replace_all(Food_part_raw, "mature leafe", "mature leaf"),
  #putting moss, stick, and bulb? into other because of low n
  Food_part_raw = str_replace_all(Food_part_raw, "stick|moss|bulb", "other"),
  Food_part_raw = str_replace_all(Food_part_raw, "trash", "human_foods"),
  Food_part_raw = str_trim(Food_part_raw)
) %>%

#also want to distinguish between mature and young leaves + ripe fruit
mutate(
  state = case_when(
    str_detect(Food_part_raw, "unripe") ~ "unripe",
    str_detect(Food_part_raw, "ripe") ~ "ripe",
    str_detect(Food_part_raw, "young|immature") ~ "young",
    str_detect(Food_part_raw, "mature") ~ "mature",

```

```

    TRUE ~ NA_character_
  ),
  Food_part_clean = str_remove_all(
    Food_part_raw,
    "young|immature|mature|ripe|unripe"
  ),
  Food_part_clean = str_trim(Food_part_clean)) %>%

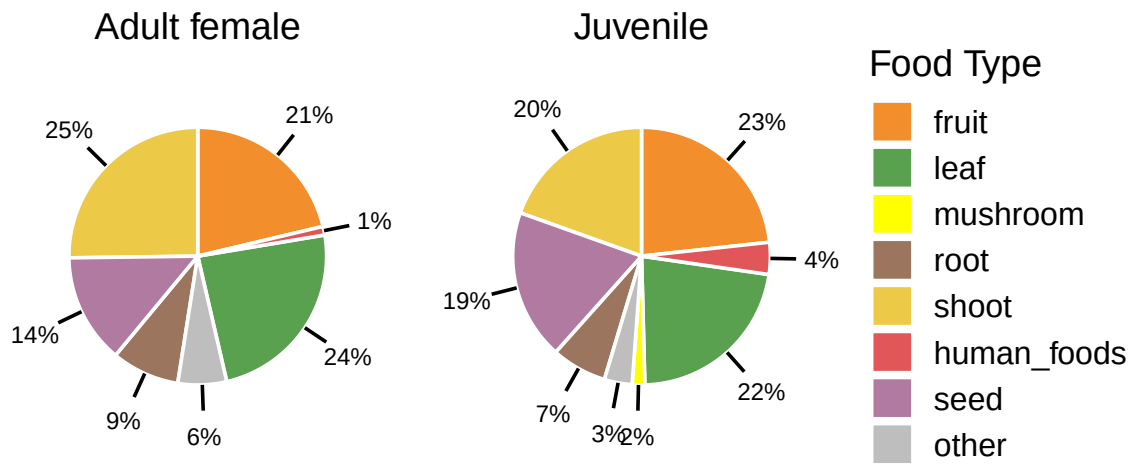
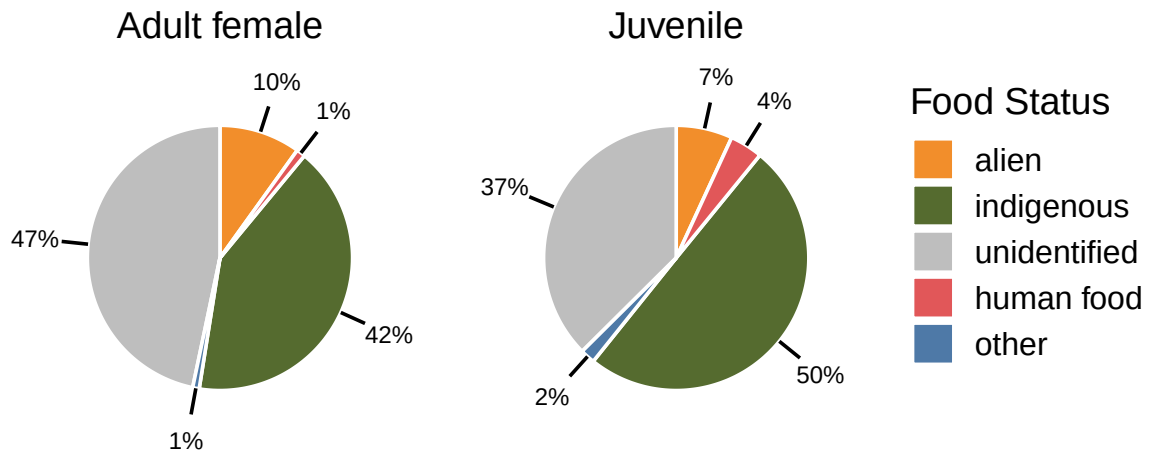
mutate(
  duration_part = Duration_sec
)

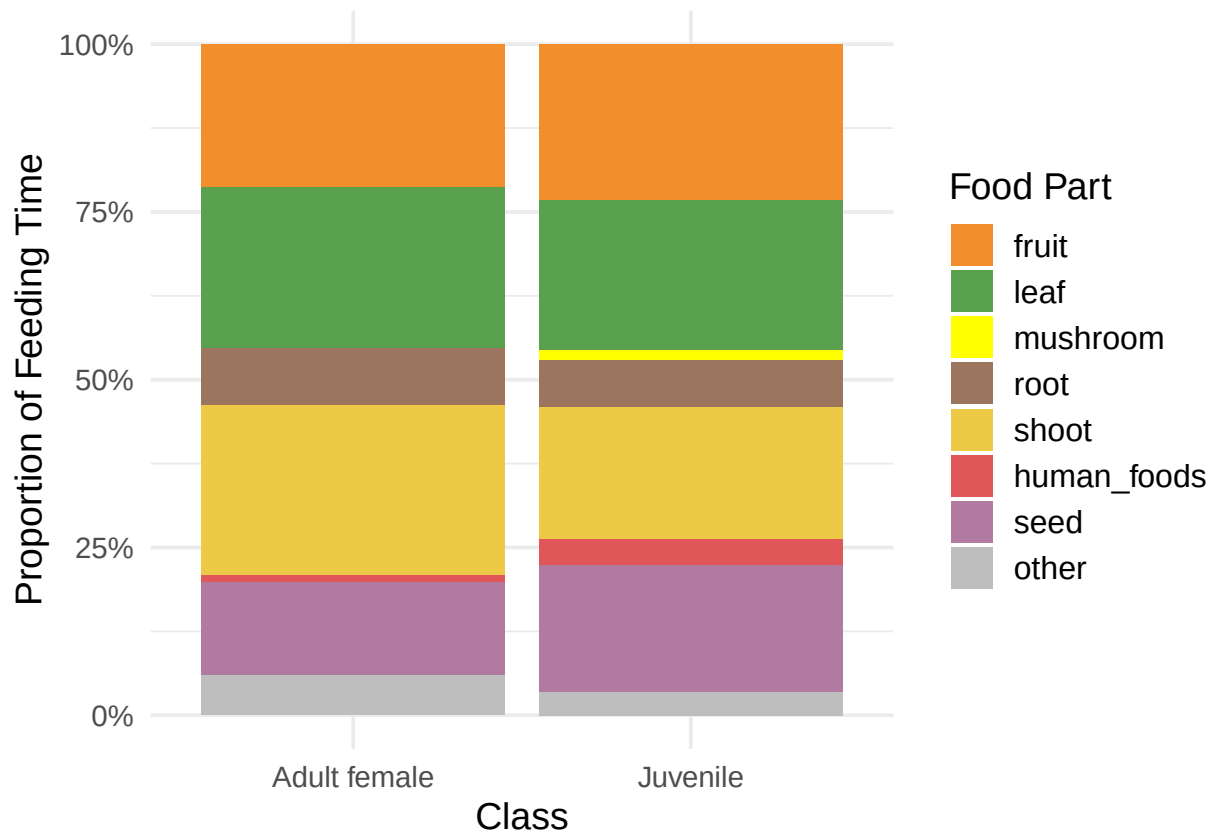
unique(clean_food_part$Food_part_clean)
#end with 12 unique food parts
#####

#fixing some "other" for status
clean_food_part = clean_food_part %>%
  mutate(Status = if_else(str_detect(Food_part_clean, "human_foods"), "human food", Status)) %>%
  mutate(Status = if_else(str_detect(Food_part_clean, "mushroom"), "other", Status)) %>%
  mutate(Status = if_else(str_detect(Food_part_clean, "insect"), "other", Status))

```

Exploratory





```
## # A tibble: 15 x 3
## # Groups:   Sex_Age [2]
##   Sex_Age      Food_part_clean percent
##   <chr>      <fct>          <dbl>
## 1 Adult female fruit            21.3
## 2 Adult female human_foods       1.1
## 3 Adult female leaf             24.0
## 4 Adult female other             6.1
## 5 Adult female root             8.5
## 6 Adult female seed            13.8
## 7 Adult female shoot           25.2
## 8 Juvenile    fruit            23.3
## 9 Juvenile    human_foods       4.0
##10 Juvenile    leaf             22.3
##11 Juvenile    mushroom          1.6
##12 Juvenile    other             3.5
##13 Juvenile    root             7.0
##14 Juvenile    seed            18.8
##15 Juvenile    shoot           19.5
```

```
indiv_food_part_percentage = clean_food_part %>%
  group_by(Focal_ID, Sex_Age, Food_part_clean) %>%
  summarise(total_time = sum(duration_part, na.rm = TRUE), .groups = "drop") %>%
  group_by(Focal_ID) %>%
  mutate(prop_time = total_time / sum(total_time)) %>%
  ungroup() %>%
```



```

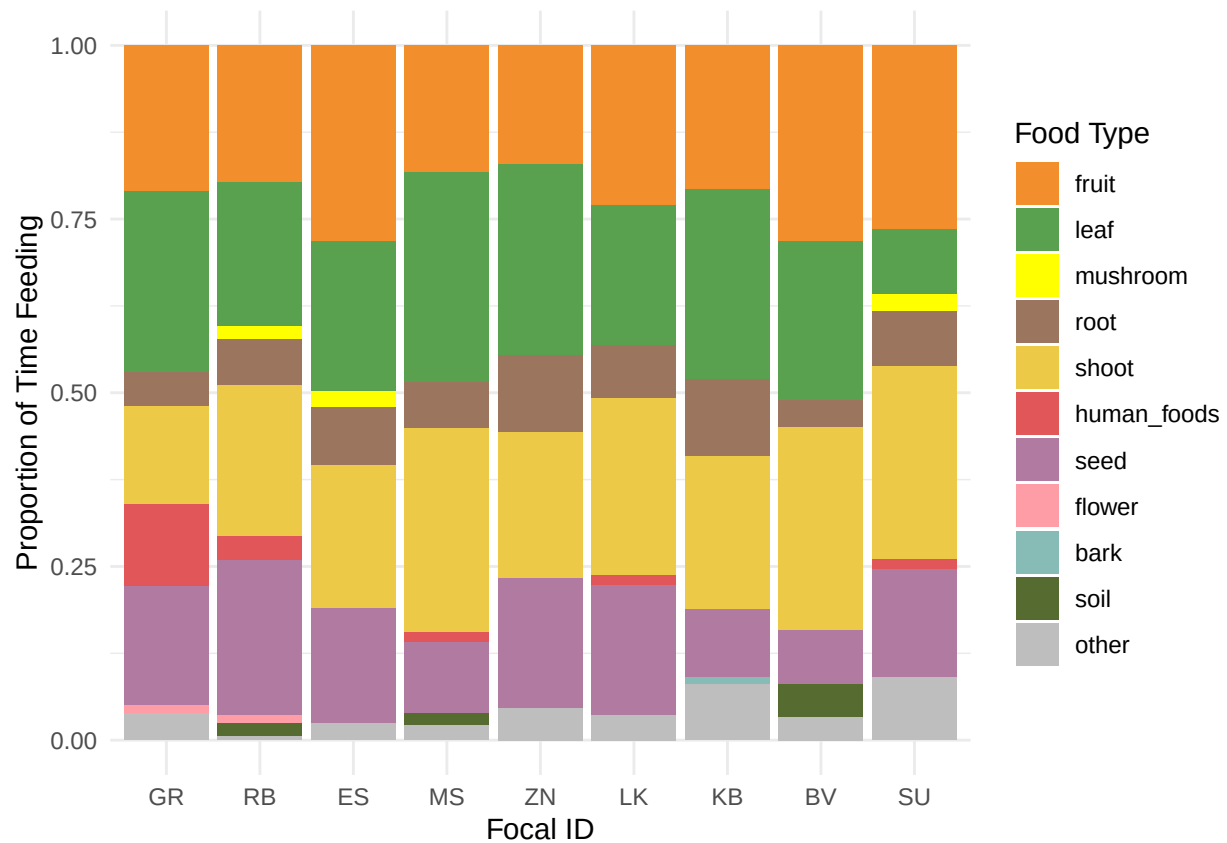
mutate(Food_part_clean = ifelse(prop_time < 0.01, "other", Food_part_clean)) %>%
group_by(Focal_ID, Sex_Age, Food_part_clean) %>%
summarise(total_time = sum(total_time), .groups = "drop") %>%
group_by(Focal_ID) %>%
mutate(prop_time = total_time / sum(total_time),
       percent = prop_time * 100) %>%
ungroup()

indiv_food_part_percentage$Food_part_clean = factor(
  indiv_food_part_percentage$Food_part_clean,
  levels = c(
    "fruit", "leaf", "mushroom", "root", "insect", "shoot",
    "human_foods", "seed", "flower", "bark", "soil", "other"))
indiv_food_part_percentage$Focal_ID = factor(indiv_food_part_percentage$Focal_ID,
  levels = c("GR", "RB", "ES", "MS", "ZN", "LK", "KB", "BV", "SU"))

#final stacked: individual variation of food type
stackedbar_3 =
  ggplot(indiv_food_part_percentage, aes(x = Focal_ID, y = percent, fill = Food_part_clean)) +
  geom_col(position = "fill") +
  labs(x = "Individual", y = "Proportion of Time Feeding", fill = "Food Part") +
  scale_fill_manual(values = c(
    "#F28E2B", # = fruit
    "#59A14F", # = leaves
    "yellow", # = mushroom
    "#9C755F", # = roots
    "#EDC948", # = shoots
    "#E15759", # = human foods
    "#B07AA1", # = seeds
    "#FF9DA7", # = flower
    "#86BCB5", # = soil
    "darkolivegreen", # = bark
    "grey" # = other
  )) +
  labs(fill = "Food Type",
       x = "Focal ID") +
  theme_minimal()

print(stackedbar_3)

```



```
#save plot
ggsave(
  filename = "stacked3.jpg",
  plot = last_plot(),
  width = 7,
  height = 5,
  units = "in",
  dpi = 300
)
# final results table per individual

food_order = c(
  "fruit", "leaf", "shoot", "seed", "mushroom", "root", "insect", "human_foods",
  "bark", "flower", "soil", "other")

#reset without rounding
indiv_food_part_table = indiv_food_part_percentage %>%
  select(Focal_ID, Food_part_clean, Sex_Age, percent) %>%
  pivot_wider(names_from = Food_part_clean, values_from = percent) %>%
  mutate(across(where(is.numeric), ~ round(.x, 1))) %>%
  replace(is.na(.), 0) %>%
  select(Focal_ID, Sex_Age, any_of(food_order)) %>%
  rename_with(~ paste0(str_to_title(.x), " (%)"),
    -c(Focal_ID, Sex_Age)) %>%
  rename(Class = Sex_Age) %>%
  arrange(Class)
```

```

print(indiv_food_part_table)

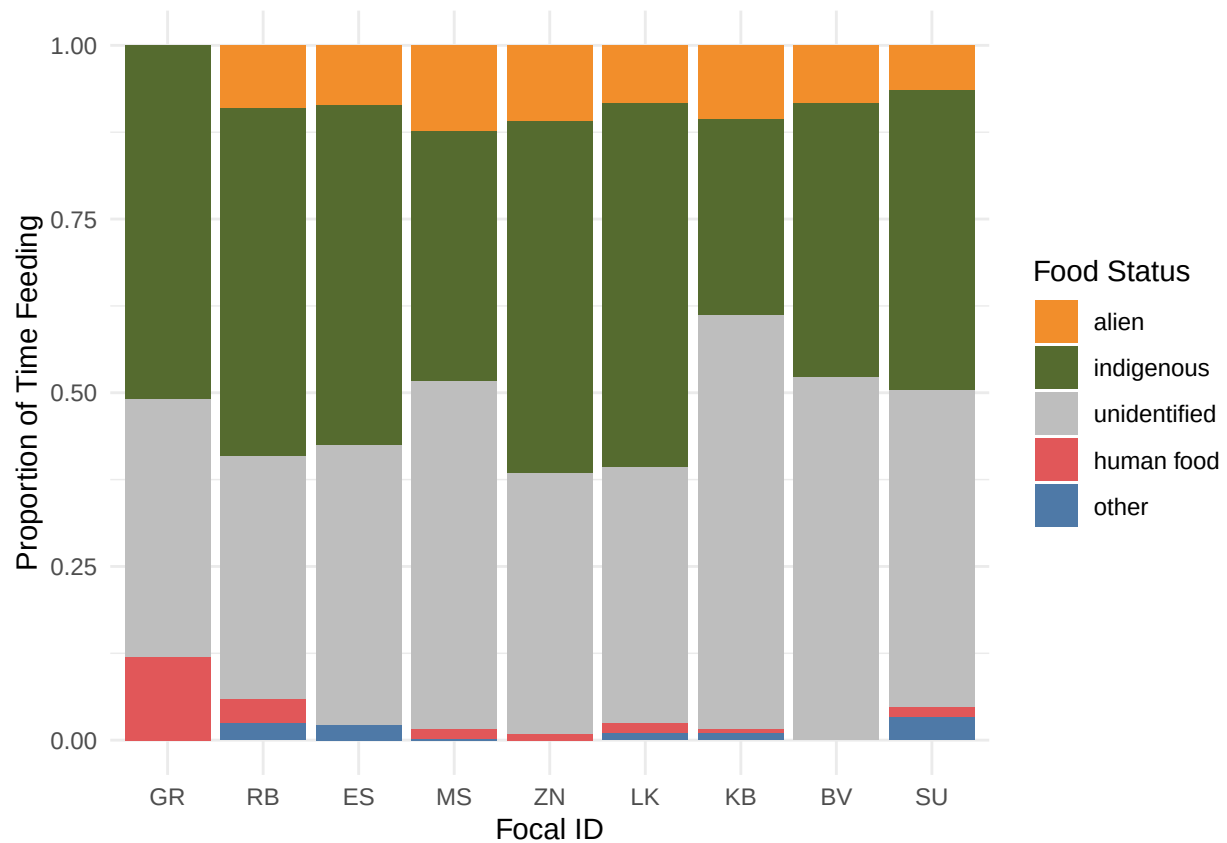
## # A tibble: 9 x 13
##   Focal_ID Class   `Fruit (%)` `Leaf (%)` `Shoot (%)` `Seed (%)` `Mushroom (%)`
##   <fct>      <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 BV        Adult f~      28.2      22.9      29.2       7.8       0
## 2 KB        Adult f~      20.7      27.3      21.9       9.8       0
## 3 LK        Adult f~      23.1      20.1      25.4      18.6       0
## 4 MS        Adult f~      18.2      30.3      29.3      10.3       0
## 5 SU        Adult f~      26.5       9.3      27.7      15.5      2.4
## 6 ZN        Adult f~      17.1      27.6      20.9      18.8       0
## 7 ES        Juvenile      28.2      21.7      20.6      16.5      2.2
## 8 GR        Juvenile      20.9      26.1      14        17.1       0
## 9 RB        Juvenile      19.7      20.8      21.7      22.3      1.8
## # i 6 more variables: `Root (%)` <dbl>, `Human_foods (%)` <dbl>,
## #   `Bark (%)` <dbl>, `Flower (%)` <dbl>, `Soil (%)` <dbl>, `Other (%)` <dbl>
##### repeating for indig #####
indiv_food_stat_percentage = clean_food_part %>%
  group_by(Focal_ID, Sex_Age, Status) %>%
  summarise(total_time = sum(duration_part, na.rm = TRUE), .groups = "drop") %>%
  group_by(Focal_ID) %>%
  mutate(prop_time = total_time / sum(total_time)) %>%
  ungroup() %>%
  group_by(Focal_ID, Sex_Age, Status) %>%
  summarise(total_time = sum(total_time), .groups = "drop") %>%
  group_by(Focal_ID) %>%
  mutate(prop_time = total_time / sum(total_time),
         percent = prop_time * 100) %>%
  ungroup()

indiv_food_stat_percentage$Focal_ID = factor(indiv_food_stat_percentage$Focal_ID,
  levels = c("GR", "RB", "ES", "MS", "ZN", "LK", "KB", "BV", "SU"))

indiv_food_stat_percentage$Status = factor(
  indiv_food_stat_percentage$Status,
  levels = c("alien", "indigenous", "unidentified", "human food", "other")
)

stackedbar_6 =
  ggplot(indiv_food_stat_percentage, aes(x = Focal_ID, y = percent, fill = Status)) +
  geom_col(position = "fill") +
  labs(x = "Individual", y = "Proportion of Time Feeding", fill = "Catagories") +
  scale_fill_manual(values = c(
    "#F28E2B",
    "darkolivegreen",
    "grey",
    "#E15759",
    "#4E79A7"
  )) +
  labs(fill = "Food Status",
       x = "Focal ID") +
  theme_minimal()
print(stackedbar_6)

```



```
#save plot
ggsave(
  filename = "stacked6.jpg",
  plot = last_plot(),
  width = 7,
  height = 5,
  units = "in",
  dpi = 300
)

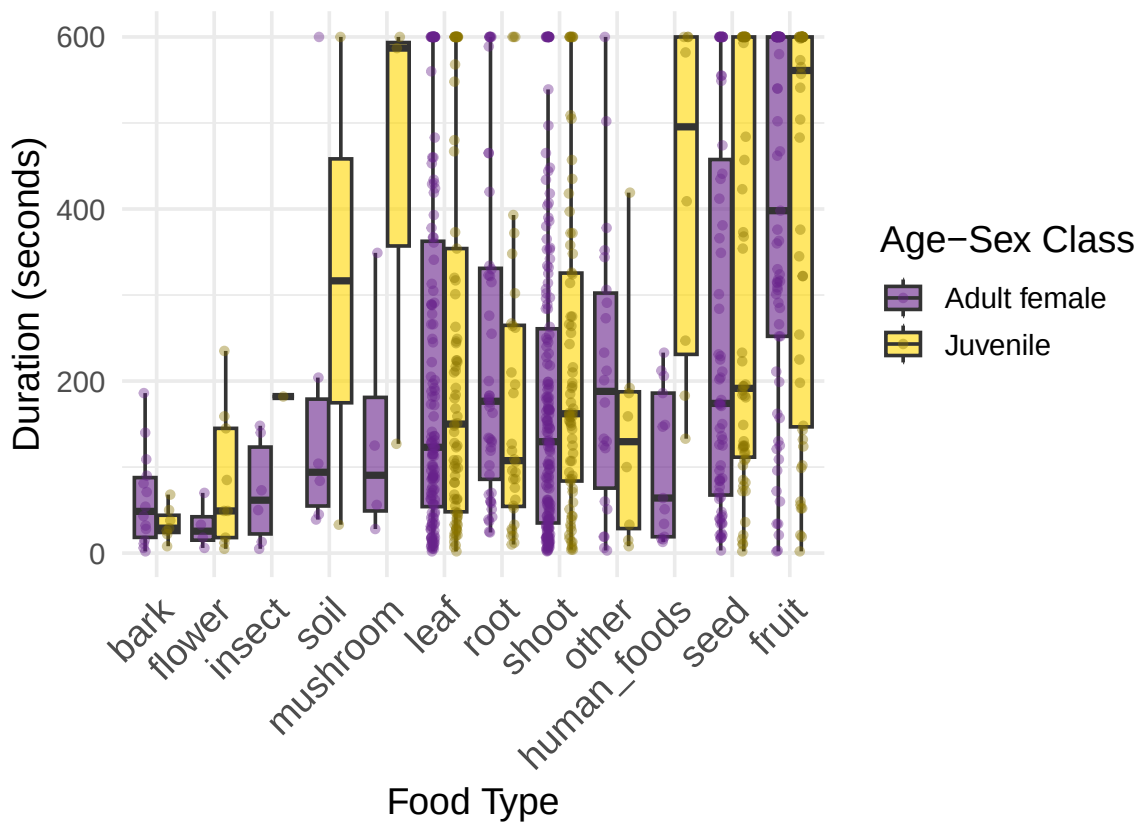
#final boxplot: variation in feeding bout duration ordered by median

food_order = clean_food_part %>%
  group_by(Food_part_clean) %>%
  summarise(med = median(duration_part, na.rm = TRUE)) %>%
  arrange(med) %>%
  pull(Food_part_clean)
clean_food_part$Food_part_clean = factor(
  clean_food_part$Food_part_clean,
  levels = food_order
)
boxplot_4 =
ggplot(clean_food_part,
  aes(x = Food_part_clean, y = duration_part, fill = Sex_Age)) +
  geom_boxplot(alpha = 0.6, outlier.shape = NA,
    position = position_dodge(0.8)) +
  geom_jitter(
```

```

aes(color = Sex_Age),
position = position_jitterdodge(
  jitter.width = 0.15,
  dodge.width = 0.8
),
alpha = 0.4,
size = 1
) +
scale_fill_manual(values = c("darkorchid4", "gold"))+
scale_color_manual(values = c("darkorchid4", "gold4"))+
labs(
  x = "Food Type",
  y = "Duration (seconds)",
  fill = "Age-Sex Class",
  color = "Age-Sex Class"
) +
theme_minimal(base_size = 14) +
theme(
  axis.text.x = element_text(size = 14, angle = 45, hjust = 1),
  plot.title = element_text(size = 16)
) +
theme(
  plot.margin = margin(t = 10, r = 20, b = 10, l = 20)
)
print(boxplot_4)

```



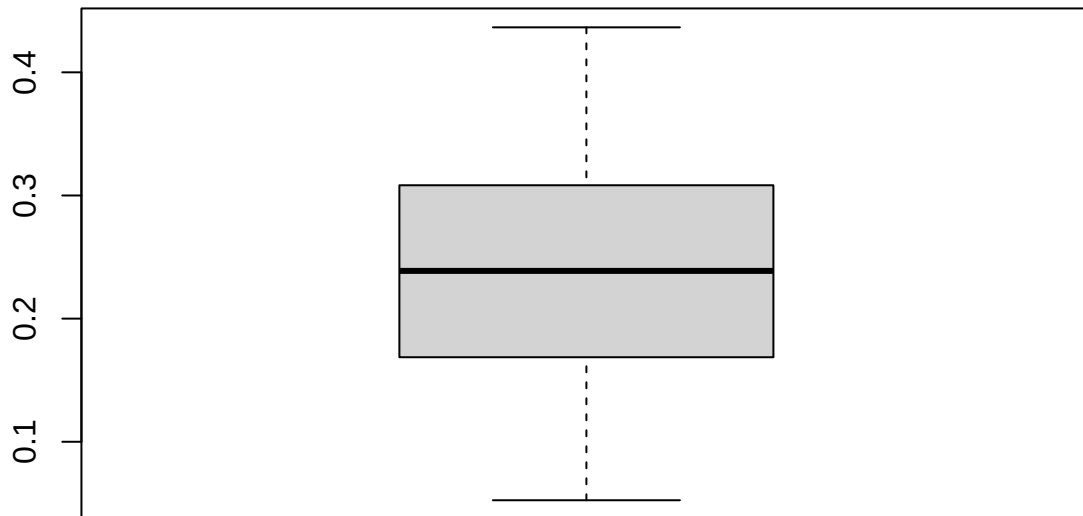
```
#save plot
ggsave(
  filename = "boxplot4.jpg",
  plot = last_plot(),
  width = 13,
  height = 7,
  units = "in",
  dpi = 300
)
```

Analyses

```
## # A tibble: 9 x 7
##   Focal_ID Sex_Age      total_time indigenous_time alien_time prop_indigenous
##   <chr>    <chr>          <dbl>          <dbl>      <dbl>      <dbl>
## 1 BV      Adult female    12850          5079.     1068.     0.395
## 2 ES      Juvenile        32759.        16028.     2834.     0.489
## 3 GR      Juvenile        18441.         9406.        0         0.510
## 4 KB      Adult female    23944.         6724.     2570.     0.281
## 5 LK      Adult female    18326.         9619.     1519.     0.525
## 6 MS      Adult female    24672.         8872.     3051.     0.360
## 7 RB      Juvenile        32454.        16291.     2918.     0.502
## 8 SU      Adult female    16780.         7227.     1093.     0.431
## 9 ZN      Adult female    27904.        14155.     3038.     0.507
## # i 1 more variable: prop_alien <dbl>

## Permutation test for adonis under reduced model
## Permutation: free
## Number of permutations: 999
##
## adonis2(formula = stat_dist_matrix ~ Sex_Age, data = stat_diet_matrix)
##           Df SumOfSqs      R2      F Pr(>F)
## Model      1 0.048438 0.18583 1.5977 0.233
## Residual   7 0.212225 0.81417
## Total      8 0.260663 1.00000

## Analysis of Variance Table
##
## Response: Distances
##           Df  Sum Sq  Mean Sq F value Pr(>F)
## Groups      1 0.000258 0.0002579  0.0278 0.8724
## Residuals    7 0.065003 0.0092862
```

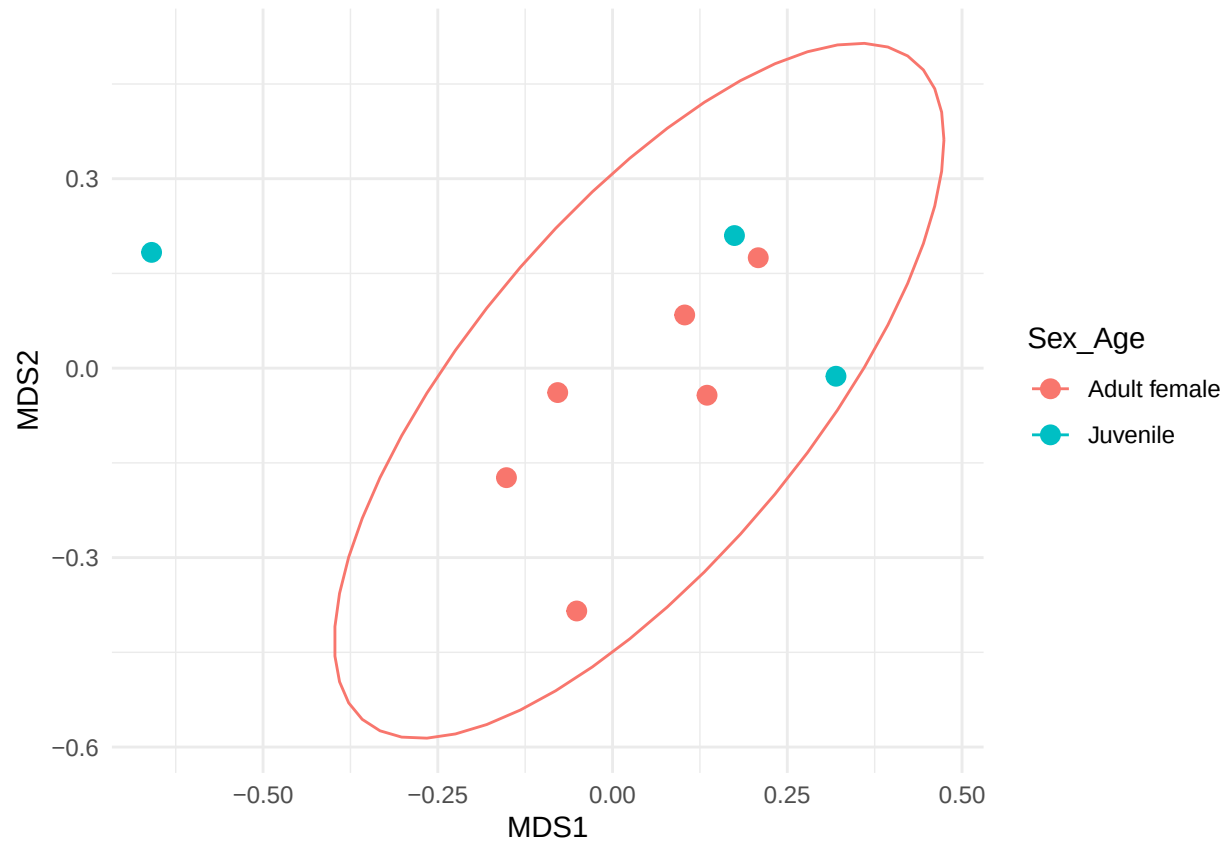


```
## Square root transformation
## Wisconsin double standardization
## Run 0 stress 0.02092866
## Run 1 stress 0.02092867
## ... Procrustes: rmse 1.671847e-05  max resid 3.208814e-05
## ... Similar to previous best
## Run 2 stress 0.02092868
## ... Procrustes: rmse 3.415816e-05  max resid 5.884086e-05
## ... Similar to previous best
## Run 3 stress 0.0209287
## ... Procrustes: rmse 4.93106e-05  max resid 9.412042e-05
## ... Similar to previous best
## Run 4 stress 0.02092866
## ... Procrustes: rmse 6.816664e-06  max resid 1.310667e-05
## ... Similar to previous best
## Run 5 stress 0.02092865
## ... New best solution
## ... Procrustes: rmse 4.171794e-05  max resid 8.769885e-05
## ... Similar to previous best
## Run 6 stress 0.02092866
## ... Procrustes: rmse 4.362678e-05  max resid 7.58619e-05
## ... Similar to previous best
## Run 7 stress 0.02092868
## ... Procrustes: rmse 5.518734e-05  max resid 0.0001130614
## ... Similar to previous best
## Run 8 stress 0.02092867
```

```

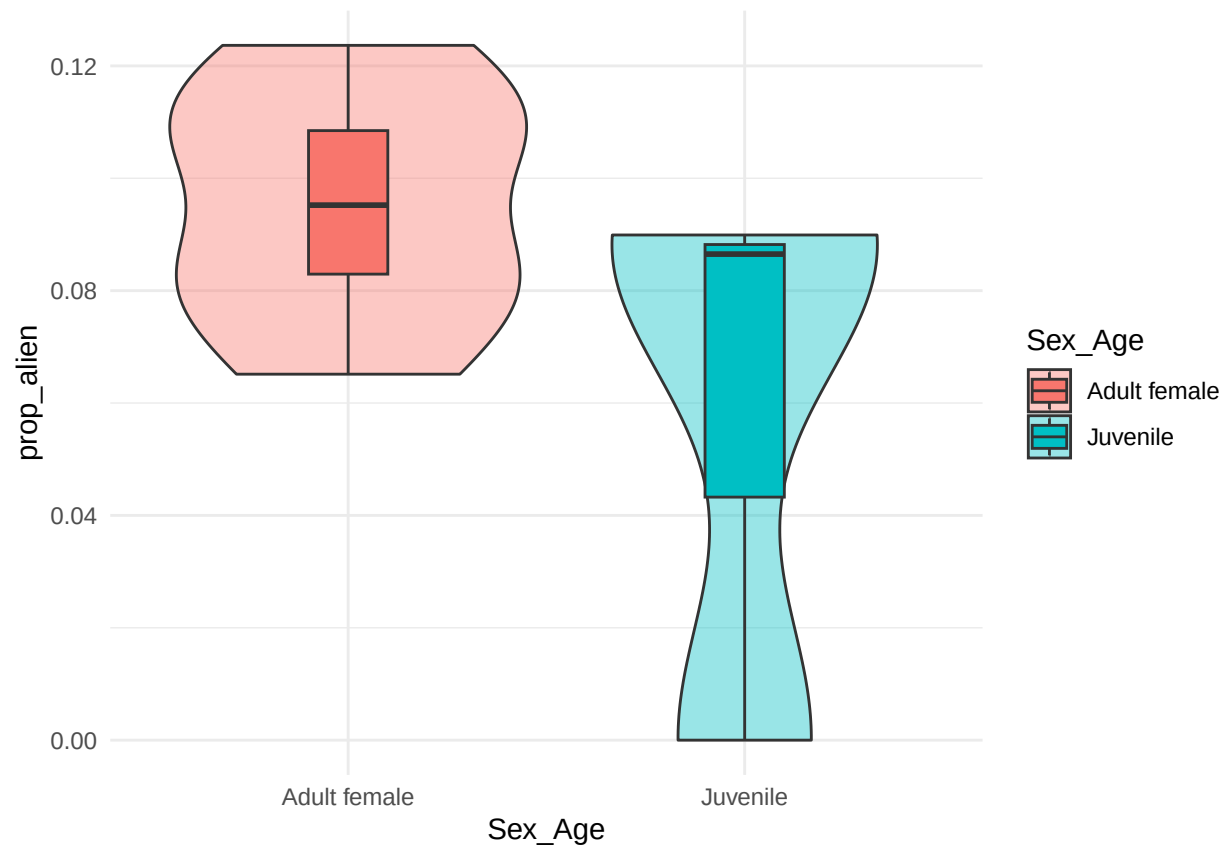
## ... Procrustes: rmse 5.058968e-05  max resid 0.000105053
## ... Similar to previous best
## Run 9 stress 0.02092867
## ... Procrustes: rmse 5.056182e-05  max resid 0.000104802
## ... Similar to previous best
## Run 10 stress 0.06066866
## Run 11 stress 0.06066862
## Run 12 stress 0.02092867
## ... Procrustes: rmse 6.697887e-05  max resid 0.0001370114
## ... Similar to previous best
## Run 13 stress 0.07642991
## Run 14 stress 0.02092865
## ... New best solution
## ... Procrustes: rmse 5.615438e-06  max resid 8.813267e-06
## ... Similar to previous best
## Run 15 stress 0.02092871
## ... Procrustes: rmse 0.0001078083  max resid 0.0002078213
## ... Similar to previous best
## Run 16 stress 0.06015448
## Run 17 stress 0.06015534
## Run 18 stress 0.06066808
## Run 19 stress 0.02092872
## ... Procrustes: rmse 0.0001113295  max resid 0.00021565
## ... Similar to previous best
## Run 20 stress 0.02092867
## ... Procrustes: rmse 5.167655e-05  max resid 9.969448e-05
## ... Similar to previous best
## *** Best solution repeated 4 times

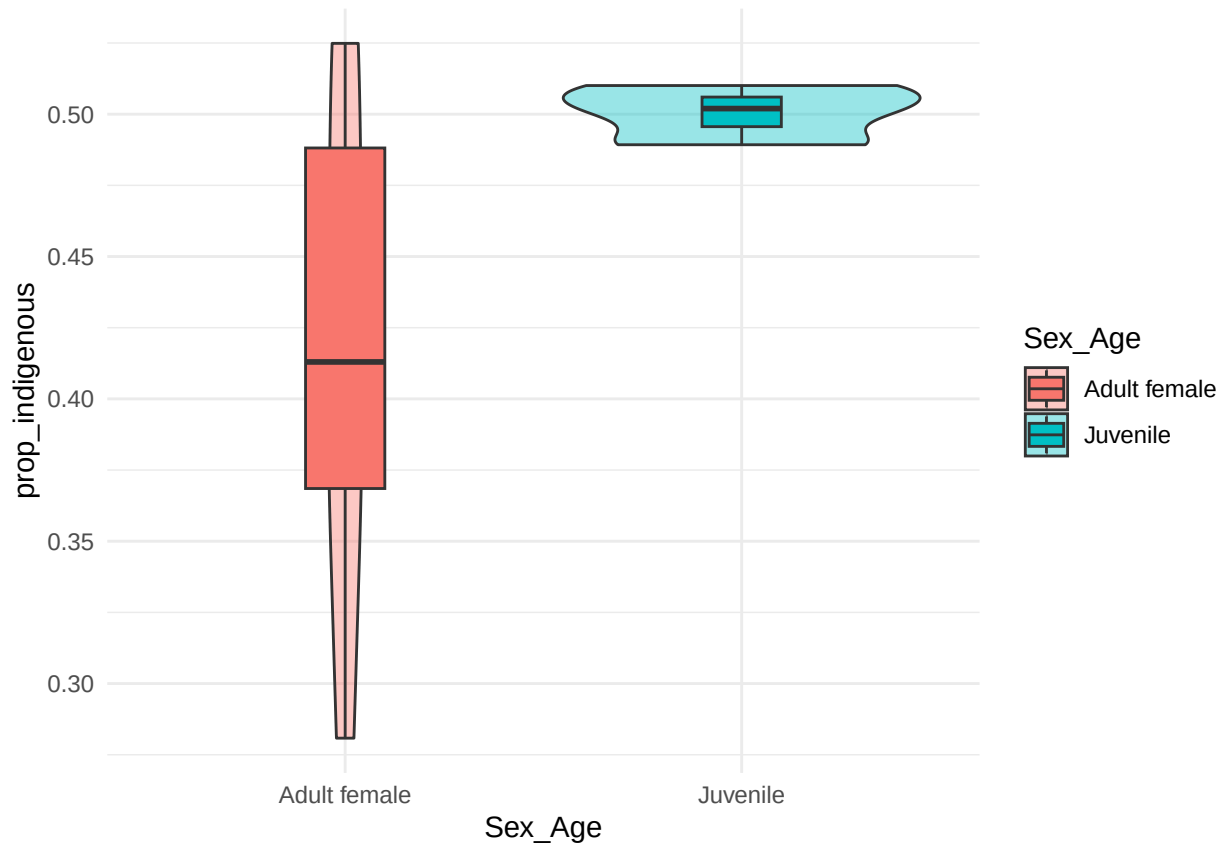
```

```
##
## Wilcoxon rank sum exact test
##
## data: prop_alien by Sex_Age
## W = 12, p-value = 0.5476
## alternative hypothesis: true location shift is not equal to 0

##
## Wilcoxon rank sum exact test
##
## data: prop_indigenous by Sex_Age
## W = 5, p-value = 0.381
## alternative hypothesis: true location shift is not equal to 0
```

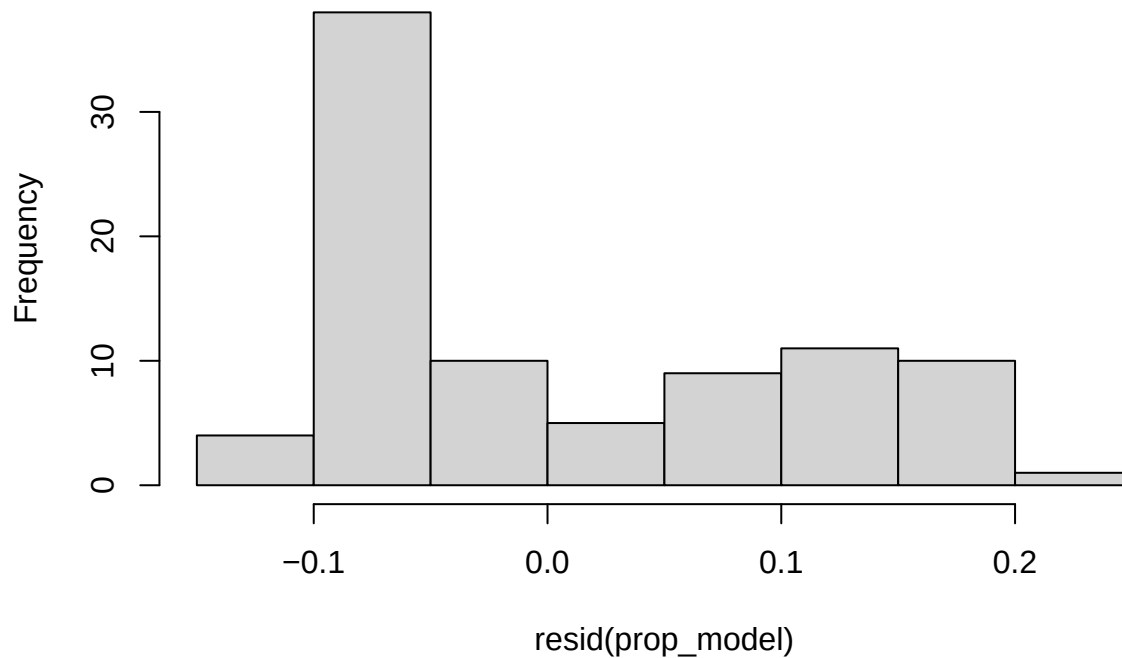




```
#bring back the low n food parts for this analysis, but will be cut later
#if they are not meaningful (bark, flower, soil, other)
indiv_food_part_percentage = clean_food_part %>%
  group_by(Focal_ID, Sex_Age, Food_part_clean) %>%
  summarise(total_time = sum(duration_part, na.rm = TRUE), .groups = "drop",
            bouts = n()) %>%
  group_by(Focal_ID) %>%
  mutate(prop_time = total_time / sum(total_time)) %>%
  ungroup() %>%
  group_by(Focal_ID, Sex_Age, Food_part_clean) %>%
  summarise(total_time = sum(total_time), bouts = sum(bouts),
            .groups = "drop") %>%
  group_by(Focal_ID) %>%
  mutate(prop_time = total_time / sum(total_time),
         percent = prop_time * 100) %>%
  ungroup()

#is my data normal? to use t-test/MANOVA (parametric)?
prop_model = lm(prop_time ~ Sex_Age, data = indiv_food_part_percentage)
hist(resid(prop_model)) # not normal
```

Histogram of resid(prop_model)



PERMANOVA for composition

```
part_diet_matrix = indiv_food_part_percentage %>%
  group_by(Focal_ID, Sex_Age, Food_part_clean) %>%
  summarise(prop_time = sum(prop_time), .groups = "drop") %>%
  tidyr::pivot_wider(names_from = Food_part_clean,
                     values_from = prop_time,
                     values_fill = 0)

part_dist_matrix = vegdist(part_diet_matrix[, -c(1,2)], method = "bray")

adonis2(part_dist_matrix ~ Sex_Age, data = part_diet_matrix)
```

```
## Permutation test for adonis under reduced model
## Permutation: free
## Number of permutations: 999
##
## adonis2(formula = part_dist_matrix ~ Sex_Age, data = part_diet_matrix)
##           Df SumOfSqs      R2      F Pr(>F)
## Model      1  0.027916 0.19829 1.7313  0.131
## Residual    7  0.112873 0.80171
## Total      8  0.140789 1.00000
```

#F = 1.73, R2 = 0.20, p = 0.15

*#About 20% of variation in diet composition is explained age/sex,
#but not significant*

```

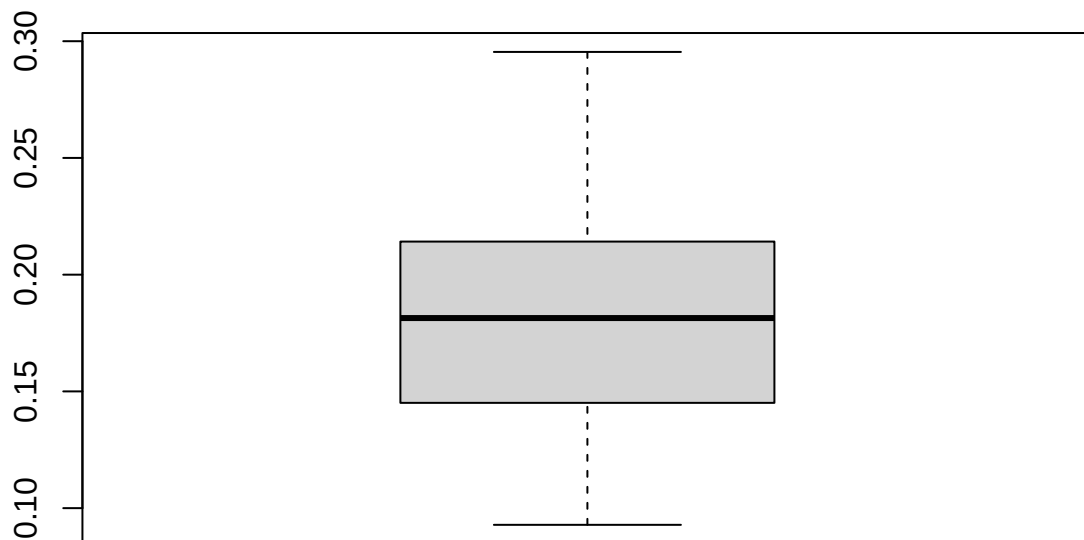
#test assumptions
betadisper(part_dist_matrix, part_diet_matrix$Sex_Age) %>%
  anova()

## Analysis of Variance Table
##
## Response: Distances
##           Df      Sum Sq    Mean Sq F value Pr(>F)
## Groups     1 0.0004729 0.00047288   0.3801 0.5571
## Residuals   7 0.0087091 0.00124415

#F = 0.38, p = 0.56
#no significant difference in dispersion between groups, PERMANOVA valid

boxplot(part_dist_matrix) #no outliers pulling distance matrix

```



```

#plotting PERMANOVA in 2D
#nto using not important
part_nmDS = metaMDS(part_diet_matrix[, -c(1,2)], distance = "bray", k = 2)

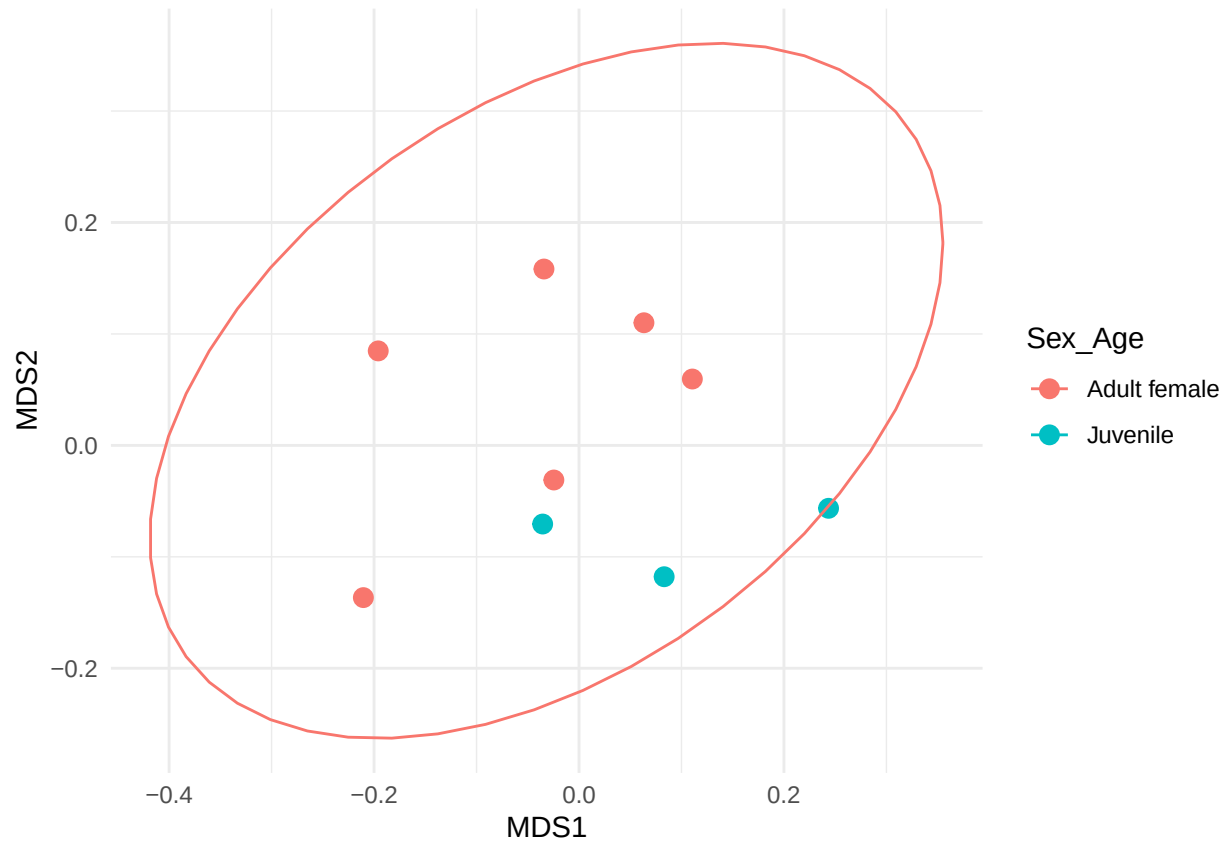
## Run 0 stress 0.08035881
## Run 1 stress 0.1687199
## Run 2 stress 0.08035881
## ... Procrustes: rmse 1.975975e-05  max resid 3.261034e-05
## ... Similar to previous best
## Run 3 stress 0.1484635
## Run 4 stress 0.08035881

```

```
## ... Procrustes: rmse 5.723977e-07  max resid 1.100266e-06
## ... Similar to previous best
## Run 5 stress 0.1460046
## Run 6 stress 0.08035881
## ... Procrustes: rmse 5.56512e-06  max resid 7.835785e-06
## ... Similar to previous best
## Run 7 stress 0.1484634
## Run 8 stress 0.1459403
## Run 9 stress 0.1460046
## Run 10 stress 0.1972752
## Run 11 stress 0.08035881
## ... Procrustes: rmse 5.215927e-06  max resid 8.491143e-06
## ... Similar to previous best
## Run 12 stress 0.08035881
## ... Procrustes: rmse 8.897183e-06  max resid 1.470288e-05
## ... Similar to previous best
## Run 13 stress 0.1448822
## Run 14 stress 0.1811541
## Run 15 stress 0.1687199
## Run 16 stress 0.1874073
## Run 17 stress 0.1459403
## Run 18 stress 0.2228105
## Run 19 stress 0.08035882
## ... Procrustes: rmse 3.557069e-05  max resid 5.586614e-05
## ... Similar to previous best
## Run 20 stress 0.08035882
## ... Procrustes: rmse 2.358462e-05  max resid 4.537849e-05
## ... Similar to previous best
## *** Best solution repeated 7 times
```

```
part_scores = as.data.frame(part_nmds$points)
part_scores$Sex_Age = part_diet_matrix$Sex_Age

ggplot(part_scores, aes(x = MDS1, y = MDS2, color = Sex_Age)) +
  geom_point(size = 3) +
  stat_ellipse(level = 0.95) +
  theme_minimal()
```



```
#do I scrap wilcoxon?
# no, keep

##### Wilcoxon by food type #####

#fruit wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "fruit"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 8, p-value = 0.9048
## alternative hypothesis: true location shift is not equal to 0
#Feeding duration on fruit did not differ significantly between AF and J
#W = 8, p = 0.90

#leaf wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "leaf"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
```

```

## W = 11, p-value = 0.7143
## alternative hypothesis: true location shift is not equal to 0
#Feeding duration on leaves did not differ significantly between AF and J
#W = 11, p = 0.71

#insect wilcox (CUT!)
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "insect"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 2, p-value = 1
## alternative hypothesis: true location shift is not equal to 0
#W = 2, p = 1
#low bout n

#shoot wilcox *
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "shoot"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 17, p-value = 0.04762
## alternative hypothesis: true location shift is not equal to 0
#Feeding duration on shoots differed significantly between AF and J
#W = 17, p = 0.048 (slightly significant)

#human_foods wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "human_foods"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 0, p-value = 0.09524
## alternative hypothesis: true location shift is not equal to 0
#Feeding duration on trash did not differ significantly between AF and J
#W = 0, p = 0.095

#root wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "root"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 11, p-value = 0.7143

```



```

## alternative hypothesis: true location shift is not equal to 0
#Feeding duration on roots did not differ significantly between AF and J
#W = 11, p = 0.71

#seed wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "seed"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 4, p-value = 0.2619
## alternative hypothesis: true location shift is not equal to 0
#Feeding duration on seeds did not differ significantly between AF and J
#W = 4, p = 0.26

#mushroom wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "mushroom"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 2, p-value = 0.8
## alternative hypothesis: true location shift is not equal to 0
#W = 2, p = 0.8
#Feeding duration on mushrooms did not differ significantly between AF and J

#bark wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "bark"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 8, p-value = 0.381
## alternative hypothesis: true location shift is not equal to 0
#W = 8, p = 0.381
#Feeding duration on bark did not differ significantly between AF and J

#flower wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "flower"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 0, p-value = 0.2
## alternative hypothesis: true location shift is not equal to 0

```

```

#Feeding duration on flowers did not differ significantly between AF and J
#W = 0, p = 0.2
#cut

#soil wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "soil"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 4, p-value = 1
## alternative hypothesis: true location shift is not equal to 0

#Feeding duration on soil did not differ significantly between AF and J
#W = 4, p = 1
#maybe cut?

#other wilcox
wilcox.test(prop_time ~ Sex_Age, data = subset(indiv_food_part_percentage,
                                                Food_part_clean == "other"))

##
## Wilcoxon rank sum exact test
##
## data: prop_time by Sex_Age
## W = 7, p-value = 0.8571
## alternative hypothesis: true location shift is not equal to 0

#Feeding duration on other did not differ significantly between AF and J
#W = 7, p = 0.86
#maybe cut due to not really meaningful

#####
wilcox_results = data.frame(
  Food_type = c("Fruit", "Leaves", "Seeds", "Mushrooms", "Roots",
               "Shoots", "Human foods", "Bark", "Flowers", "Soil", "Other"),

  W = c(8, 11, 4, 2, 11, 17, 0, 8, 0, 4, 7),

  p_value = c(0.90, 0.71, 0.26, 0.80, 0.71,
              0.048, 0.095, 0.381, 0.20, 1.00, 0.86)
)

wilcox_results$Interpretation = ifelse(
  wilcox_results$p_value < 0.05, "*",
  " ")
print(wilcox_results)

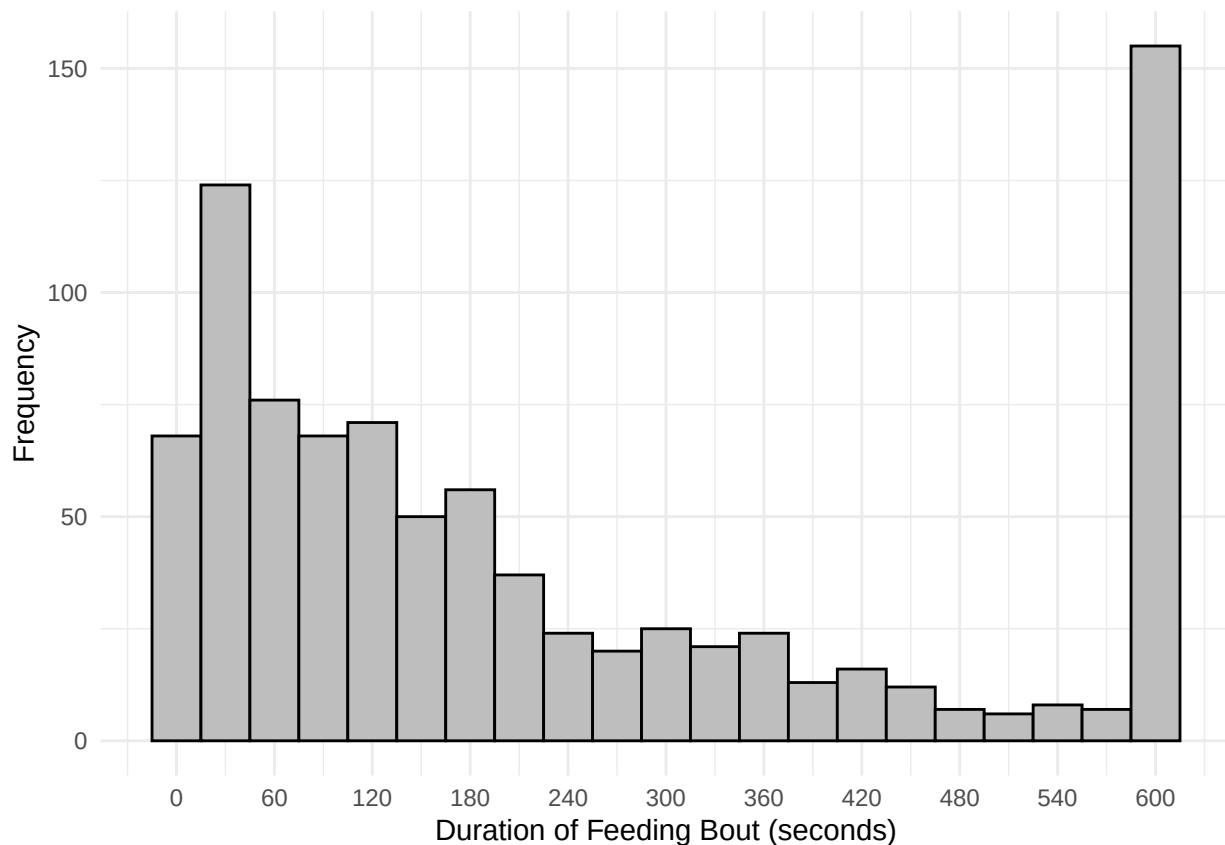
##      Food_type  W p_value Interpretation
## 1      Fruit   8  0.900
## 2     Leaves  11  0.710
## 3      Seeds   4  0.260
## 4  Mushrooms   2  0.800

```

```
## 5      Roots 11  0.710
## 6      Shoots 17  0.048
## 7 Human foods 0  0.095
## 8      Bark 8   0.381
## 9      Flowers 0  0.200
## 10     Soil 4   1.000
## 11     Other 7   0.860
```

choosing GLMM distribution type

```
ggplot(clean_food_part, aes(x = duration_part)) +
  geom_histogram(binwidth = 30,
                 fill = "grey",
                 color = "black") +
  labs(x = "Duration of Feeding Bout (seconds)", y = "Frequency") +
  scale_x_continuous(breaks = seq(0, 600, by = 60)) +
  theme_minimal()
```



*#showing gamma distribution (right skew, non-neg), but high number of 10mins
#which is not traditional gamma distrib...*

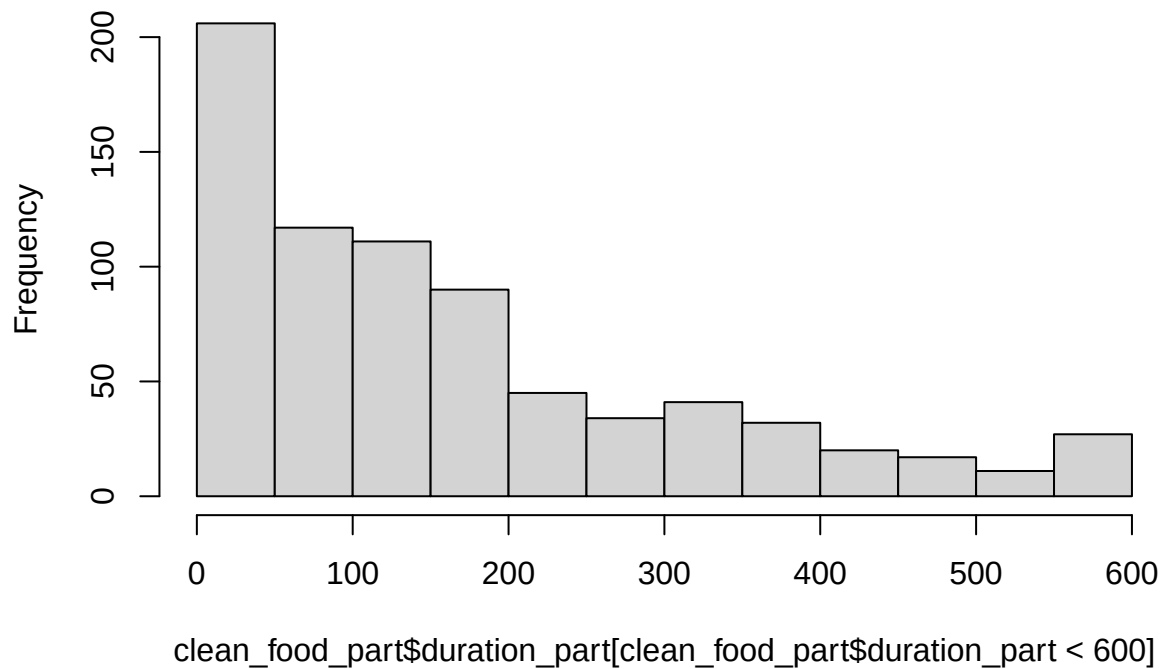
```
mean(clean_food_part$duration_part == 600)
```

```
## [1] 0.1475225
```

*#about 14.8% of the data are 600s, which is a lot, but these are all the "in-prog"
#to "in-prog" bouts, which means its capping at this number.*

```
hist(clean_food_part$duration_part[clean_food_part$duration_part < 600])
```

ogram of clean_food_part\$duration_part[clean_food_part\$duration_pa



*#without the 600s, the distribution looks more like a gamma distribution,
#with a right skew and a long tail.*

*#I think Gamma distribution is still appropriate, but there is a limitation for
#capping at 600s, which is something to note in the discussion.*

```
#glmm with gamma + log
m_gam = glmmTMB(duration_part ~ Sex_Age + Food_part_clean + Sex_Age:Food_part_clean +
  (1 | Focal_ID),
  family = Gamma(link = "log"),
  data = clean_food_part
)
summary(m_gam)
```

```
## Family: Gamma ( log )
## Formula:
## duration_part ~ Sex_Age + Food_part_clean + Sex_Age:Food_part_clean +
## (1 | Focal_ID)
## Data: clean_food_part
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##  11388.8  11513.3  -5668.4   11336.8      862
##
## Random effects:
##
```

```

## Conditional model:
## Groups Name Variance Std.Dev.
## Focal_ID (Intercept) 0.01104 0.1051
## Number of obs: 888, groups: Focal_ID, 9
##
## Dispersion estimate for Gamma family (sigma^2): 0.934
##
## Conditional model:
## Estimate Std. Error z value Pr(>|z|)
## (Intercept) 4.0798 0.2632 15.501 < 2e-16
## Sex_AgeJuvenile -0.5238 0.4557 -1.150 0.250297
## Food_part_cleanflower -0.6053 0.5510 -1.098 0.272014
## Food_part_cleaninsect 0.1344 0.4728 0.284 0.776155
## Food_part_cleansoil 1.1640 0.4765 2.443 0.014573
## Food_part_cleanmushroom 0.8606 0.5510 1.562 0.118336
## Food_part_cleanleaf 1.2739 0.2720 4.683 2.83e-06
## Food_part_cleanroot 1.3809 0.2971 4.649 3.34e-06
## Food_part_cleanshoot 1.1167 0.2701 4.135 3.55e-05
## Food_part_cleanother 1.2121 0.3308 3.664 0.000249
## Food_part_cleanhuman_foods 0.5623 0.3738 1.504 0.132472
## Food_part_cleanseed 1.4604 0.2854 5.116 3.12e-07
## Food_part_cleanfruit 1.9004 0.2854 6.658 2.77e-11
## Sex_AgeJuvenile:Food_part_cleanflower 1.4759 0.7372 2.002 0.045280
## Sex_AgeJuvenile:Food_part_cleaninsect 1.4704 1.1397 1.290 0.196989
## Sex_AgeJuvenile:Food_part_cleansoil 1.0001 0.9149 1.093 0.274326
## Sex_AgeJuvenile:Food_part_cleanmushroom 1.6382 0.8670 1.890 0.058815
## Sex_AgeJuvenile:Food_part_cleanleaf 0.6065 0.4693 1.292 0.196271
## Sex_AgeJuvenile:Food_part_cleanroot 0.2538 0.5023 0.505 0.613329
## Sex_AgeJuvenile:Food_part_cleanshoot 0.7115 0.4702 1.513 0.130275
## Sex_AgeJuvenile:Food_part_cleanother 0.1947 0.5999 0.324 0.745571
## Sex_AgeJuvenile:Food_part_cleanhuman_foods 1.9496 0.6263 3.113 0.001852
## Sex_AgeJuvenile:Food_part_cleanseed 0.6494 0.4840 1.342 0.179721
## Sex_AgeJuvenile:Food_part_cleanfruit 0.5419 0.4855 1.116 0.264410
##
## (Intercept) ***
## Sex_AgeJuvenile
## Food_part_cleanflower
## Food_part_cleaninsect
## Food_part_cleansoil *
## Food_part_cleanmushroom
## Food_part_cleanleaf ***
## Food_part_cleanroot ***
## Food_part_cleanshoot ***
## Food_part_cleanother ***
## Food_part_cleanhuman_foods
## Food_part_cleanseed ***
## Food_part_cleanfruit ***
## Sex_AgeJuvenile:Food_part_cleanflower *
## Sex_AgeJuvenile:Food_part_cleaninsect
## Sex_AgeJuvenile:Food_part_cleansoil
## Sex_AgeJuvenile:Food_part_cleanmushroom .
## Sex_AgeJuvenile:Food_part_cleanleaf
## Sex_AgeJuvenile:Food_part_cleanroot
## Sex_AgeJuvenile:Food_part_cleanshoot

```

```

## Sex_AgeJuvenile:Food_part_cleanother
## Sex_AgeJuvenile:Food_part_cleanhuman_foods **
## Sex_AgeJuvenile:Food_part_cleanseed
## Sex_AgeJuvenile:Food_part_cleanfruit
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

#log used for interpretability of coefficients as proportional changes in
#feeding duration, which is more meaningful for this type of data. This can be
#interpreted as the percentage change in feeding duration associated with each
#predictor.

#glmm with simplified interaction
m_simp = glmmTMB(duration_part ~ Sex_Age + Food_part_clean + (1|Focal_ID),
                  family = lognormal(),
                  data = clean_food_part)
summary(m_simp)

## Family: lognormal ( log )
## Formula:      duration_part ~ Sex_Age + Food_part_clean + (1 | Focal_ID)
## Data: clean_food_part
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##  11547.7  11619.5   -5758.8   11517.7      873
##
## Random effects:
##
## Conditional model:
##   Groups   Name      Variance Std.Dev.
## Focal_ID (Intercept) 0.0121   0.11
## Number of obs: 888, groups: Focal_ID, 9
##
## Dispersion parameter for lognormal family (): 559
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      4.95908    0.18320  27.069 < 2e-16 ***
## Sex_AgeJuvenile    0.04387    0.08987   0.488 0.625450
## Food_part_cleanflower 0.12127    0.27240   0.445 0.656187
## Food_part_cleaninsect 0.34968    0.32260   1.084 0.278386
## Food_part_cleansoil   0.66903    0.27922   2.396 0.016574 *
## Food_part_cleanmushroom 0.88632    0.28066   3.158 0.001589 **
## Food_part_cleanleaf   0.62271    0.17481   3.562 0.000368 ***
## Food_part_cleanroot   0.74612    0.18289   4.080 4.51e-05 ***
## Food_part_cleanshoot  0.55360    0.17434   3.175 0.001496 **
## Food_part_cleanother  0.55689    0.20749   2.684 0.007275 **
## Food_part_cleanhuman_foods 0.67004    0.21937   3.054 0.002255 **
## Food_part_cleanseed   0.81607    0.17919   4.554 5.26e-06 ***
## Food_part_cleanfruit   1.08868    0.17865   6.094 1.10e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

#glmm with lognormal distrib
m_lognorm = glmmTMB(duration_part ~ Sex_Age * Food_part_clean + (1|Focal_ID),
                    family = lognormal(),

```

```

                                data = clean_food_part)
summary(m_lognorm)

## Family: lognormal ( log )
## Formula:          duration_part ~ Sex_Age * Food_part_clean + (1 | Focal_ID)
## Data: clean_food_part
##
##           AIC          BIC      logLik -2*log(L)  df.resid
##    11545.6    11670.1    -5746.8    11493.6      862
##
## Random effects:
##
## Conditional model:
##   Groups   Name      Variance Std.Dev.
##   Focal_ID (Intercept) 0.01202  0.1096
## Number of obs: 888, groups:  Focal_ID, 9
##
## Dispersion parameter for lognormal family (): 534
##
## Conditional model:
##                                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)                        4.93346    0.21775   22.657 < 2e-16
## Sex_AgeJuvenile                     0.05341    0.36390    0.147  0.88332
## Food_part_cleanflower                -0.08955    0.45594   -0.196  0.84430
## Food_part_cleaninsect                 0.26735    0.36751    0.727  0.46695
## Food_part_cleansoil                   0.68540    0.32587    2.103  0.03544
## Food_part_cleanmushroom               0.67736    0.38211    1.773  0.07628
## Food_part_cleanleaf                   0.62621    0.21378    2.929  0.00340
## Food_part_cleanroot                   0.86715    0.22219    3.903 9.51e-05
## Food_part_cleanshoot                   0.51840    0.21263    2.438  0.01476
## Food_part_cleanother                   0.65113    0.24752    2.631  0.00852
## Food_part_cleanhuman_foods             0.40183    0.28121    1.429  0.15303
## Food_part_cleanseed                   0.81743    0.22052    3.707  0.00021
## Food_part_cleanfruit                  1.14103    0.21870    5.217 1.82e-07
## Sex_AgeJuvenile:Food_part_cleanflower  0.30409    0.59284    0.513  0.60800
## Sex_AgeJuvenile:Food_part_cleaninsect  0.83480    0.69770    1.197  0.23150
## Sex_AgeJuvenile:Food_part_cleansoil   -0.06801    0.62806   -0.108  0.91377
## Sex_AgeJuvenile:Food_part_cleanmushroom 0.54923    0.55449    0.991  0.32192
## Sex_AgeJuvenile:Food_part_cleanleaf   -0.01695    0.36581   -0.046  0.96305
## Sex_AgeJuvenile:Food_part_cleanroot   -0.27030    0.38203   -0.708  0.47923
## Sex_AgeJuvenile:Food_part_cleanshoot   0.10924    0.36557    0.299  0.76507
## Sex_AgeJuvenile:Food_part_cleanother   -0.30531    0.45064   -0.678  0.49808
## Sex_AgeJuvenile:Food_part_cleanhuman_foods 1.01118    0.43705    2.314  0.02069
## Sex_AgeJuvenile:Food_part_cleanseed   -0.00878    0.37316   -0.024  0.98123
## Sex_AgeJuvenile:Food_part_cleanfruit   -0.12407    0.37218   -0.333  0.73886
##
## (Intercept)                        ***
## Sex_AgeJuvenile
## Food_part_cleanflower
## Food_part_cleaninsect
## Food_part_cleansoil                  *
## Food_part_cleanmushroom              .
## Food_part_cleanleaf                  **
## Food_part_cleanroot                  ***

```

```

## Food_part_cleanshoot *
## Food_part_cleanother **
## Food_part_cleanhuman_foods
## Food_part_cleanseed ***
## Food_part_cleanfruit ***
## Sex_AgeJuvenile:Food_part_cleanflower
## Sex_AgeJuvenile:Food_part_cleaninsect
## Sex_AgeJuvenile:Food_part_cleansoil
## Sex_AgeJuvenile:Food_part_cleanmushroom
## Sex_AgeJuvenile:Food_part_cleanleaf
## Sex_AgeJuvenile:Food_part_cleanroot
## Sex_AgeJuvenile:Food_part_cleanshoot
## Sex_AgeJuvenile:Food_part_cleanother
## Sex_AgeJuvenile:Food_part_cleanhuman_foods *
## Sex_AgeJuvenile:Food_part_cleanseed
## Sex_AgeJuvenile:Food_part_cleanfruit
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

#glmm with gaussian
m_gauss = lmer(duration_part ~ Sex_Age * Food_part_clean + (1|Focal_ID),
               data = clean_food_part)
summary(m_gauss)

```

```

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: duration_part ~ Sex_Age * Food_part_clean + (1 | Focal_ID)
## Data: clean_food_part
##
## REML criterion at convergence: 11631.6
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.1145 -0.7597 -0.2706  0.7313  2.3412
##
## Random effects:
## Groups Name Variance Std.Dev.
## Focal_ID (Intercept) 922.6 30.37
## Residual 37701.8 194.17
## Number of obs: 888, groups: Focal_ID, 9
##
## Fixed effects:
##
## Estimate Std. Error df t value
## (Intercept) 44.2944 53.6806 509.6120 0.825
## Sex_AgeJuvenile -6.7769 92.8358 516.7369 -0.073
## Food_part_cleanflower 0.3181 111.0524 863.9512 0.003
## Food_part_cleaninsect 27.6298 95.2164 862.3274 0.290
## Food_part_cleansoil 137.4042 95.2636 862.4459 1.442
## Food_part_cleanmushroom 102.8075 110.8401 863.6995 0.928
## Food_part_cleanleaf 171.5249 54.7725 863.0735 3.132
## Food_part_cleanroot 187.7345 59.6914 859.3913 3.145
## Food_part_cleanshoot 138.3134 54.2982 863.3027 2.547
## Food_part_cleanother 154.7925 66.6424 861.5330 2.323
## Food_part_cleanhuman_foods 58.5739 75.1813 862.7782 0.779
## Food_part_cleanseed 213.0596 57.4648 863.6133 3.708

```



```

## Food_part_cleanfruit          353.8159    57.4190  863.1199    6.162
## Sex_AgeJuvenile:Food_part_cleanflower    51.1917   148.2882  863.6568    0.345
## Sex_AgeJuvenile:Food_part_cleaninsect    107.5146   229.2041  861.8086    0.469
## Sex_AgeJuvenile:Food_part_cleansoil      144.7745   182.8960  860.8683    0.792
## Sex_AgeJuvenile:Food_part_cleanmushroom   290.3009   174.1468  862.1177    1.667
## Sex_AgeJuvenile:Food_part_cleanleaf      19.6770    94.3091  861.8976    0.209
## Sex_AgeJuvenile:Food_part_cleanroot     -47.6951   101.0168  861.9057   -0.472
## Sex_AgeJuvenile:Food_part_cleanshoot     42.5102    94.2708  862.0781    0.451
## Sex_AgeJuvenile:Food_part_cleanother    -45.8756   120.6700  859.5032   -0.380
## Sex_AgeJuvenile:Food_part_cleanhuman_foods 326.3547   126.0550  862.9306    2.589
## Sex_AgeJuvenile:Food_part_cleanseed      38.6006    97.2714  863.1902    0.397
## Sex_AgeJuvenile:Food_part_cleanfruit     11.9062    97.5851  862.3107    0.122
##                                     Pr(>|t|)
## (Intercept)                        0.409675
## Sex_AgeJuvenile                    0.941835
## Food_part_cleanflower              0.997715
## Food_part_cleaninsect              0.771749
## Food_part_cleansoil                0.149564
## Food_part_cleanmushroom            0.353911
## Food_part_cleanleaf                0.001797 **
## Food_part_cleanroot                0.001718 **
## Food_part_cleanshoot               0.011029 *
## Food_part_cleanother               0.020426 *
## Food_part_cleanhuman_foods         0.436133
## Food_part_cleanseed                0.000223 ***
## Food_part_cleanfruit               1.1e-09 ***
## Sex_AgeJuvenile:Food_part_cleanflower 0.730015
## Sex_AgeJuvenile:Food_part_cleaninsect 0.639133
## Sex_AgeJuvenile:Food_part_cleansoil   0.428831
## Sex_AgeJuvenile:Food_part_cleanmushroom 0.095880 .
## Sex_AgeJuvenile:Food_part_cleanleaf   0.834776
## Sex_AgeJuvenile:Food_part_cleanroot   0.636939
## Sex_AgeJuvenile:Food_part_cleanshoot  0.652148
## Sex_AgeJuvenile:Food_part_cleanother  0.703910
## Sex_AgeJuvenile:Food_part_cleanhuman_foods 0.009788 **
## Sex_AgeJuvenile:Food_part_cleanseed   0.691588
## Sex_AgeJuvenile:Food_part_cleanfruit  0.902921
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

##
## Correlation matrix not shown by default, as p = 24 > 12.
## Use print(x, correlation=TRUE) or
##     vcov(x)           if you need it

AIC_comp = AIC(m_gam, m_simp, m_lognorm, m_gauss)
print(AIC_comp)

##           df      AIC
## m_gam      26 11388.83
## m_simp     15 11547.70
## m_lognorm  26 11545.56
## m_gauss    26 11683.64

```

*#The GLMM with a Gamma distribution has the lowest AIC, suggesting it is the
 #best fit for the data compared to the lognormal and Gaussian models. This
 #supports using a Gamma distribution for modeling feeding bout
 #durations, which are right-skewed and non-negative.*

```
#set ref food to fruit bc common
clean_food_part$Food_part_clean = as.factor(clean_food_part$Food_part_clean)
clean_food_part$Food_part_clean = relevel(
  clean_food_part$Food_part_clean,
  ref = "fruit"
)

### GLMM for duration ###
glmm_bout = glmmTMB(duration_part ~ Sex_Age * Food_part_clean + (1 | Focal_ID),
  family = Gamma(link = "log"),
  data = clean_food_part
)
summary(glmm_bout)
```

```
## Family: Gamma ( log )
## Formula:      duration_part ~ Sex_Age * Food_part_clean + (1 | Focal_ID)
## Data: clean_food_part
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##  11388.8  11513.3  -5668.4   11336.8      862
##
## Random effects:
##
## Conditional model:
## Groups   Name      Variance Std.Dev.
## Focal_ID (Intercept) 0.01104  0.1051
## Number of obs: 888, groups:  Focal_ID, 9
##
## Dispersion estimate for Gamma family (sigma^2): 0.934
##
## Conditional model:
##                                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)                        5.98016    0.12566   47.59 < 2e-16
## Sex_AgeJuvenile                     0.01802    0.19781    0.09  0.92742
## Food_part_cleanbark                 -1.90041    0.28541   -6.66 2.77e-11
## Food_part_cleanflower                -2.50570    0.49966   -5.01 5.31e-07
## Food_part_cleaninsect               -1.76590    0.41404   -4.27 2.00e-05
## Food_part_cleansoil                 -0.73645    0.41492   -1.77  0.07591
## Food_part_cleanmushroom             -1.03980    0.49972   -2.08  0.03746
## Food_part_cleanleaf                 -0.62653    0.14421   -4.34 1.39e-05
## Food_part_cleanroot                 -0.51950    0.18833   -2.76  0.00581
## Food_part_cleanshoot                -0.78366    0.13906   -5.64 1.75e-08
## Food_part_cleanother                -0.68837    0.23860   -2.89  0.00391
## Food_part_cleanhuman_foods          -1.33805    0.29332   -4.56 5.07e-06
## Food_part_cleanseed                 -0.44001    0.16742   -2.63  0.00858
## Sex_AgeJuvenile:Food_part_cleanbark -0.54190    0.48552   -1.12  0.26437
## Sex_AgeJuvenile:Food_part_cleanflower 0.93400    0.61105    1.53  0.12639
## Sex_AgeJuvenile:Food_part_cleaninsect 0.92850    1.06203    0.87  0.38198
## Sex_AgeJuvenile:Food_part_cleansoil  0.45819    0.81394    0.56  0.57349
```

```
## Sex_AgeJuvenile:Food_part_cleanmushroom    1.09635    0.76222    1.44    0.15033
## Sex_AgeJuvenile:Food_part_cleanleaf         0.06462    0.22777    0.28    0.77665
## Sex_AgeJuvenile:Food_part_cleanroot        -0.28799    0.28999   -0.99    0.32065
## Sex_AgeJuvenile:Food_part_cleanshoot        0.16962    0.22695    0.75    0.45483
## Sex_AgeJuvenile:Food_part_cleanother       -0.34711    0.44176   -0.79    0.43201
## Sex_AgeJuvenile:Food_part_cleanhuman_foods  1.40768    0.47428    2.97    0.00300
## Sex_AgeJuvenile:Food_part_cleanseed         0.10759    0.25483    0.42    0.67288
##
## (Intercept)                                ***
## Sex_AgeJuvenile
## Food_part_cleanbark                        ***
## Food_part_cleanflower                      ***
## Food_part_cleaninsect                      ***
## Food_part_cleansoil                        .
## Food_part_cleanmushroom                    *
## Food_part_cleanleaf                        ***
## Food_part_cleanroot                        **
## Food_part_cleanshoot                       ***
## Food_part_cleanother                       **
## Food_part_cleanhuman_foods                 ***
## Food_part_cleanseed                        **
## Sex_AgeJuvenile:Food_part_cleanbark
## Sex_AgeJuvenile:Food_part_cleanflower
## Sex_AgeJuvenile:Food_part_cleaninsect
## Sex_AgeJuvenile:Food_part_cleansoil
## Sex_AgeJuvenile:Food_part_cleanmushroom
## Sex_AgeJuvenile:Food_part_cleanleaf
## Sex_AgeJuvenile:Food_part_cleanroot
## Sex_AgeJuvenile:Food_part_cleanshoot
## Sex_AgeJuvenile:Food_part_cleanother
## Sex_AgeJuvenile:Food_part_cleanhuman_foods **
## Sex_AgeJuvenile:Food_part_cleanseed
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##### results table #####
```

```
coefs = summary(glm_bout)$coefficients$cond
glmm_results = as.data.frame(coefs)
glmm_results$term = rownames(glmm_results)
rownames(glmm_results) = NULL
glmm_results$exp_estimate = exp(glmm_results$Estimate)

glmm_results = glmm_results %>%
  mutate(
    is_interaction = str_detect(term, ":"),

    food = case_when(
      str_detect(term, "Food_part_clean") ~
        str_remove(term, ".*Food_part_clean"),
      TRUE ~ NA_character_
    ),
    Measured = case_when(
      term == "(Intercept)" ~ "Adult baseline",
      term == "Sex_AgeJuvenile" ~ "Juvenile vs Adult (baseline food)",
```

```

    is_interaction ~ paste("Juvenile vs Adult:", food),
    str_detect(term, "Food_part_clean") ~ paste("Adult:", food),
    TRUE ~ term
  )) %>%
  rename(
    Effect_size = exp_estimate,
    Std_error = `Std. Error`,
    p_value = 'Pr(>|z|)'
  ) %>%
  mutate(
    p_value = case_when(
      p_value < 0.001 ~ "<0.001",
      TRUE ~ sprintf("%.3f", p_value)
    ),
    significance = case_when(
      p_value < 0.001 ~ "***",
      p_value < 0.01 ~ "**",
      p_value < 0.05 ~ "*",
      TRUE ~ ""
    )
  ) %>%
  select(Measured, Estimate, Std_error, Effect_size, p_value, significance)
print(glm_results)

```

	Measured	Estimate	Std_error	Effect_size	p_value
## 1	Adult baseline	5.98015564	0.1256610	395.50191765	<0.001
## 2	Juvenile vs Adult (baseline food)	0.01802017	0.1978119	1.01818352	0.927
## 3	Adult: bark	-1.90040912	0.2854126	0.14950744	<0.001
## 4	Adult: flower	-2.50570252	0.4996556	0.08161824	<0.001
## 5	Adult: insect	-1.76590134	0.4140418	0.17103256	<0.001
## 6	Adult: soil	-0.73644841	0.4149189	0.47881144	0.076
## 7	Adult: mushroom	-1.03980115	0.4997202	0.35352497	0.037
## 8	Adult: leaf	-0.62652619	0.1442057	0.53444514	<0.001
## 9	Adult: root	-0.51949873	0.1883331	0.59481864	0.006
## 10	Adult: shoot	-0.78365556	0.1390580	0.45673334	<0.001
## 11	Adult: other	-0.68836698	0.2386017	0.50239582	0.004
## 12	Adult: human_foods	-1.33805029	0.2933169	0.26235669	<0.001
## 13	Adult: seed	-0.44001255	0.1674166	0.64402834	0.009
## 14	Juvenile vs Adult: bark	-0.54189521	0.4855202	0.58164487	0.264
## 15	Juvenile vs Adult: flower	0.93399738	0.6110517	2.54466086	0.126
## 16	Juvenile vs Adult: insect	0.92849603	1.0620328	2.53070021	0.382
## 17	Juvenile vs Adult: soil	0.45818641	0.8139422	1.58120373	0.573
## 18	Juvenile vs Adult: mushroom	1.09634696	0.7622221	2.99321170	0.150
## 19	Juvenile vs Adult: leaf	0.06461617	0.2277718	1.06674949	0.777
## 20	Juvenile vs Adult: root	-0.28799479	0.2899918	0.74976550	0.321
## 21	Juvenile vs Adult: shoot	0.16962190	0.2269510	1.18485677	0.455
## 22	Juvenile vs Adult: other	-0.34711140	0.4417567	0.70672659	0.432
## 23	Juvenile vs Adult: human_foods	1.40768179	0.4742844	4.08647110	0.003
## 24	Juvenile vs Adult: seed	0.10758583	0.2548257	1.11358644	0.673
##	significance				
## 1	***				
## 2					
## 3	***				
## 4	***				

```
## 5          ***
## 6
## 7          *
## 8          ***
## 9          **
## 10         ***
## 11         **
## 12         ***
## 13         **
## 14
## 15
## 16
## 17
## 18
## 19
## 20
## 21
## 22
## 23         **
## 24
```

#Interpretations:

#Adults spend more time feeding on fruit (the reference food) compared to other food types, indicated by the significant positive coefficients for the food part terms.

#Juveniles do not differ significantly from adults in feeding duration in general (non-significant coefficient), but they do significantly differ from adults in their feeding duration on human foods (garbage), as indicated by the significant interaction term. It is important to note

#####

#ggpredict is using the fitted GLMM predictions to create a new data frame with predicted values and confidence intervals for each combination of food part and age/sex class.

```
pred = ggpredict(glm_bout, terms = c("Food_part_clean", "Sex_Age"))
```

```
## You are calculating adjusted predictions on the population-level (i.e.
## `type = "fixed"`) for a *generalized* linear mixed model.
## This may produce biased estimates due to Jensen's inequality. Consider
## setting `bias_correction = TRUE` to correct for this bias.
## See also the documentation of the `bias_correction` argument.
```

```
## Some of the focal terms are of type `character`. This may lead to
## unexpected results. It is recommended to convert these variables to
## factors before fitting the model.
## The following variables are of type character: `Sex_Age`
```

```
pred$group = factor(pred$group,
                     levels = c("Adult female", "Juvenile"))
```

#It also transforms the log scale back to the original so it is more easily interpretable for a plot.

#Graphs indicating predicted feeding bout duration for each food part by

#Juveniles and AF. Error bars indicate 95% confidence intervals.

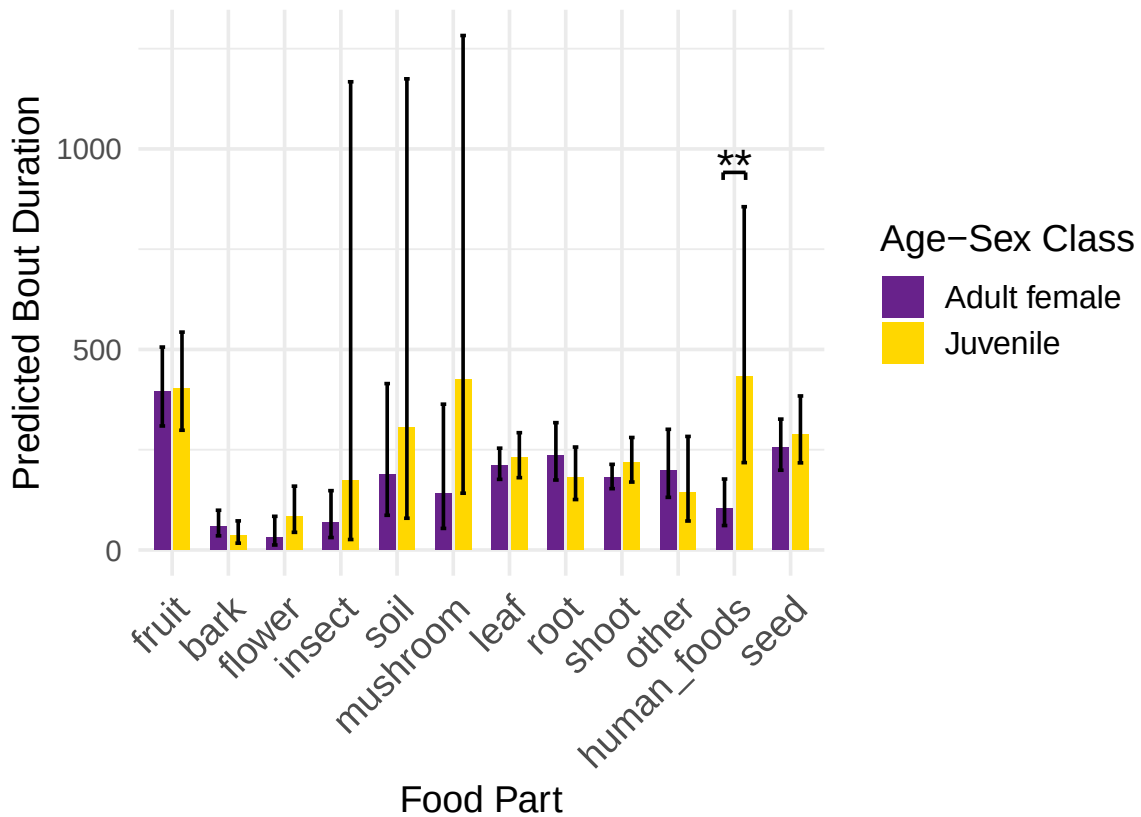
#final predicted bar plot: GLMM pred results with error bars

```
sig_data = subset(pred, x == "human_foods")

sig_bracket = data.frame(
  x = 1, # human_foods (position index)
  xstart = 11 - 0.2,
  xend = 11 + 0.2,
  y = max(sig_data$conf.high) * 1.1,
  label = "**"
)

ggplot(pred, aes(x = x, y = predicted, fill = group)) +
  geom_col(position = position_dodge(0.7), width = 0.6) +
  geom_errorbar(
    aes(ymin = conf.low, ymax = conf.high),
    position = position_dodge(0.7),
    width = 0.2
  ) +
  scale_fill_manual(values = c(
    "Adult female" = "darkorchid4",
    "Juvenile" = "gold"
  )) +
  geom_segment(
    data = sig_bracket,
    aes(x = xstart, xend = xend, y = y, yend = y),
    inherit.aes = FALSE
  ) +
  geom_segment(
    data = sig_bracket,
    aes(x = xstart, xend = xstart, y = y, yend = y * 0.98),
    inherit.aes = FALSE
  ) +
  geom_segment(
    data = sig_bracket,
    aes(x = xend, xend = xend, y = y, yend = y * 0.98),
    inherit.aes = FALSE
  ) +
  geom_text(
    data = sig_bracket,
    aes(x = (xstart + xend) / 2, y = y * 1.02, label = label),
    inherit.aes = FALSE,
    size = 6
  ) +
  scale_y_continuous() +
  labs(
    x = "Food Part",
    y = "Predicted Bout Duration",
    fill = "Age-Sex Class") +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.x = element_text(size = 14, angle = 45, hjust = 1),
```

```
axis.title.y = element_text(size = 14),
plot.title = element_text(size = 18),
legend.text = element_text(size = 12)) +
theme(
plot.margin = margin(t = 10, r = 20, b = 10, l = 20)
)
```



```
#save plot
ggsave(
  filename = "boxplot5.jpg",
  plot = last_plot(),
  width = 13,
  height = 7,
  units = "in",
  dpi = 300
)
```

```
#Assumption of Gamma = no 0s
summary(clean_food_part$duration_part)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      2.00  56.75  158.50  234.38  376.50  600.00
```

```
#Assumption of Gamma = right skewed = yes (tested earlier in workflow)
```

```
#Assumption of independence handled with random effects (1 | Focal_ID)
```

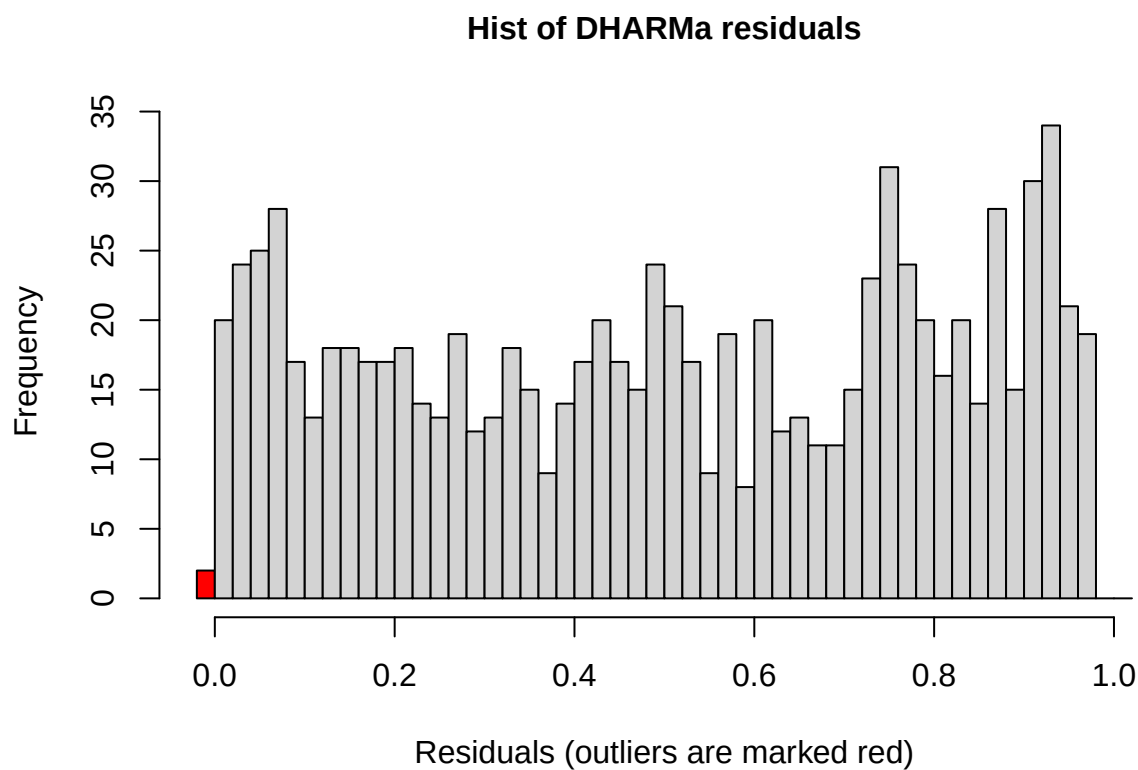
```
#Assumption that random effects covers variation and isn't extremely skewed
ranef(glm_bout)
```

```
## $Focal_ID
##      (Intercept)
## BV -0.08344606
## ES  0.01522296
## GR -0.06350045
## KB  0.12306343
## LK -0.12911390
## MS -0.03222157
## RB  0.04420691
## SU  0.04462470
## ZN  0.06890897
```

```
#yep
```

```
#Assumption that residuals aren't patterned
sim_res = simulateResiduals(glm_bout)
```

```
hist(sim_res) #few outliers
```



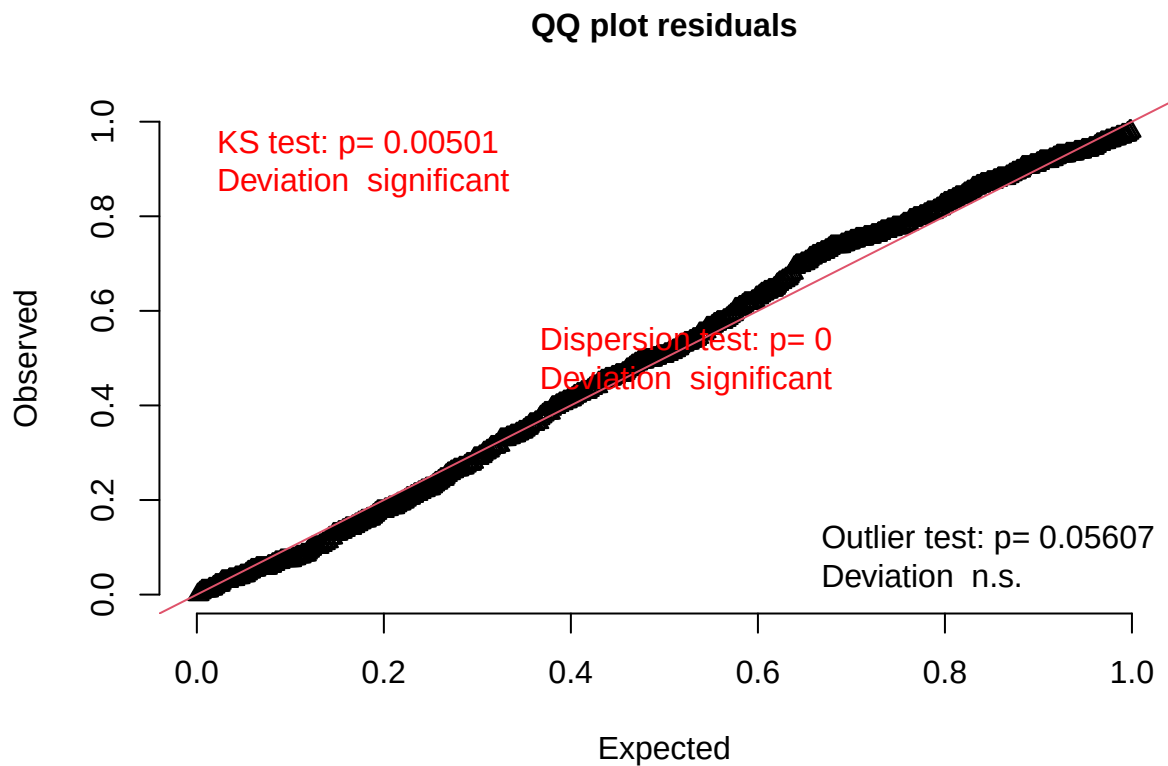
```
plotQQunif(sim_res, group = clean_food_part$Sex_Age)
```

```
## Warning in plot.window(...): "group" is not a graphical parameter
```

```
## Warning in plot.xy(xy, type, ...): "group" is not a graphical parameter
```



```
## Warning in axis(side = side, at = at, labels = labels, ...): "group" is not a
## graphical parameter
## Warning in axis(side = side, at = at, labels = labels, ...): "group" is not a
## graphical parameter
## Warning in box(...): "group" is not a graphical parameter
## Warning in title(...): "group" is not a graphical parameter
```

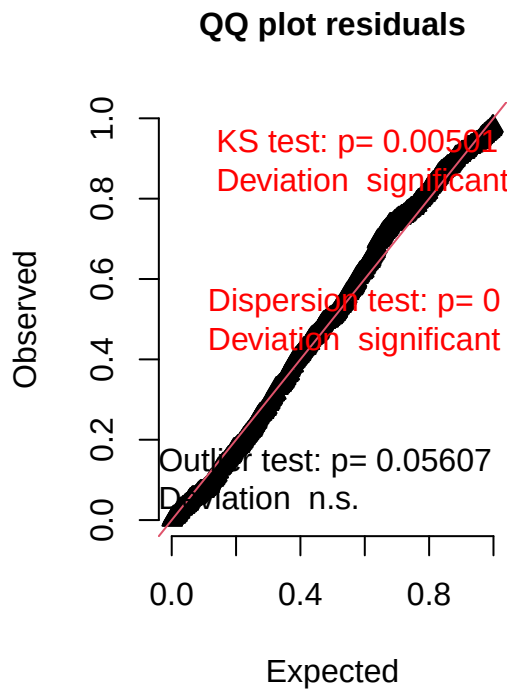


```
#QQ is not masively skewed, but there is some deviation that is stat sig
```

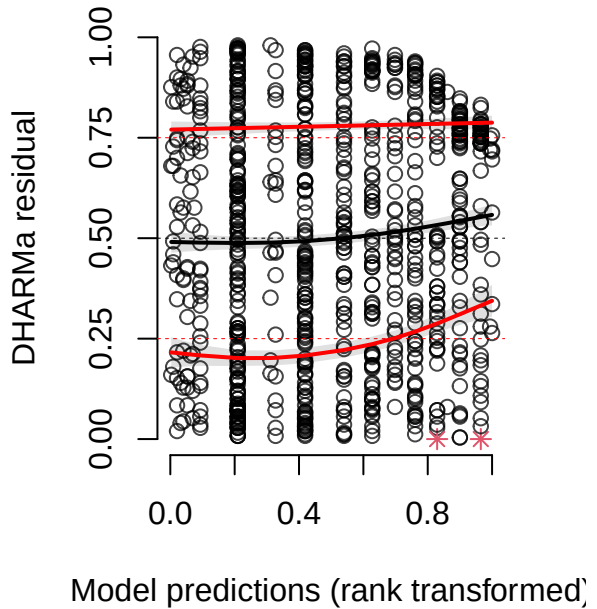
```
plot(sim_res)
```

```
## Warning in newton(lsp = lsp, X = G$X, y = G$y, Eb = G$Eb, UrS = G$UrS, L = G$L,
## : Fitting terminated with step failure - check results carefully
```

DHARMa residual

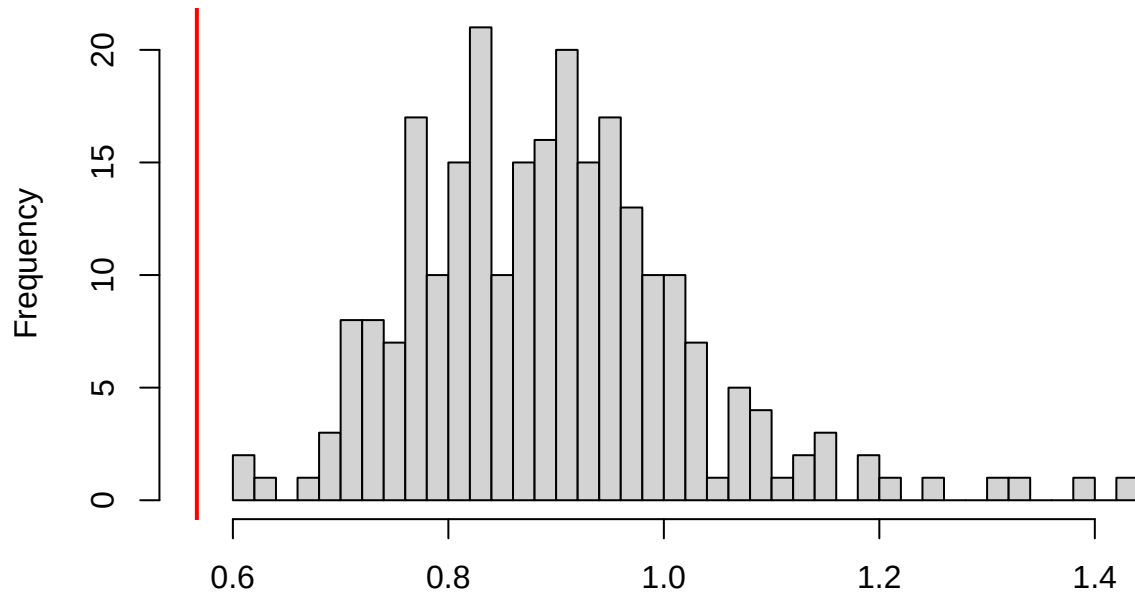


DHARMa residual vs. predicted
Quantile deviations detected (red curves)
Combined adjusted quantile test significant



```
testDispersion(sim_res)
```

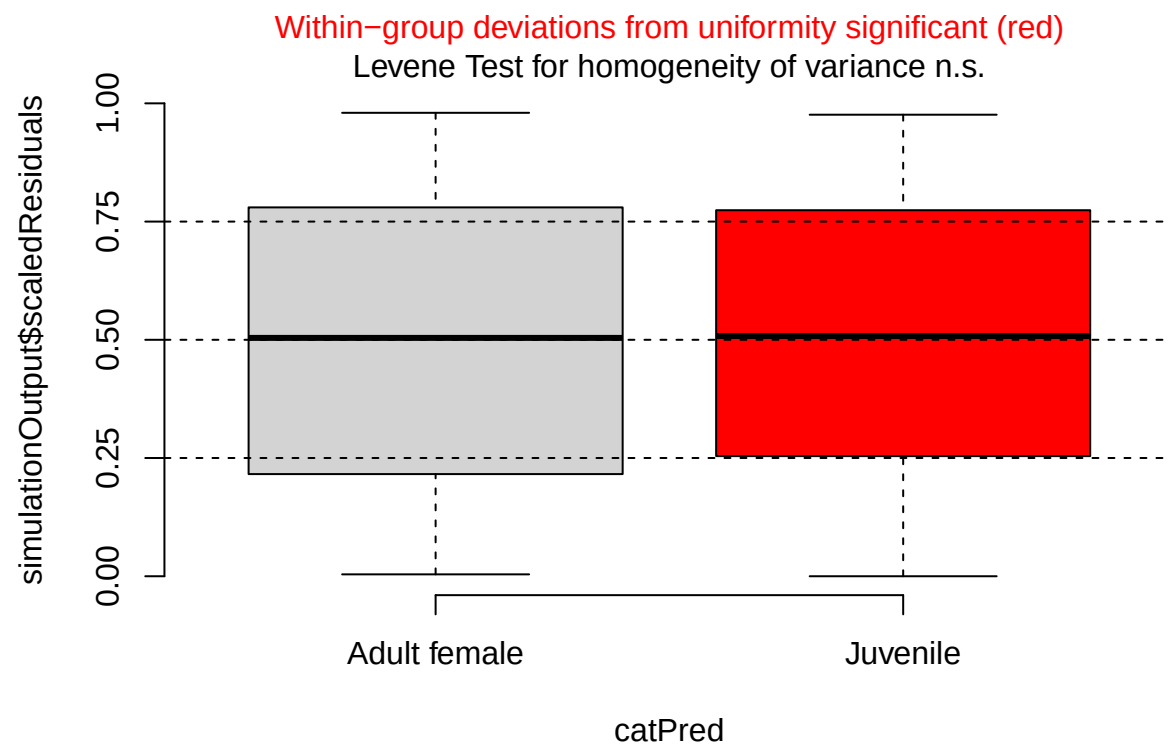
DHARMA nonparametric dispersion test via sd of residuals fitted vs. simulated



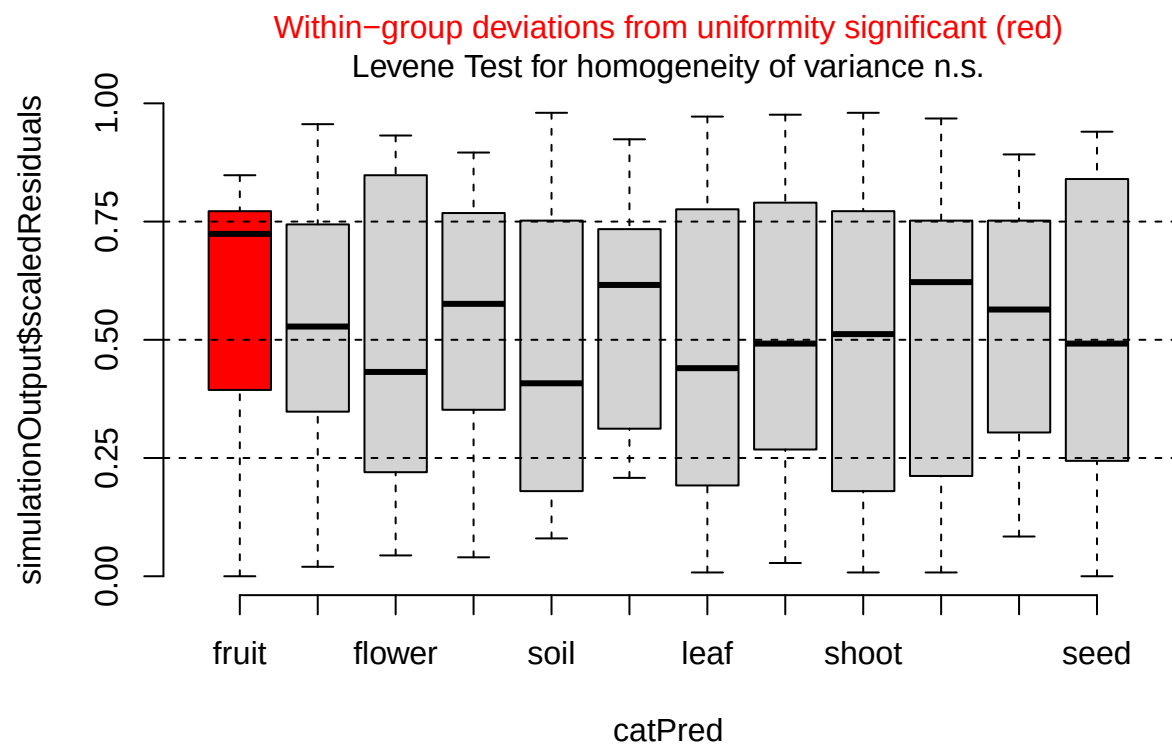
Simulated values, red line = fitted model. p-value (two.sided) = 0

```
##
## DHARMA nonparametric dispersion test via sd of residuals fitted vs.
## simulated
##
## data: simulationOutput
## dispersion = 0.63296, p-value < 2.2e-16
## alternative hypothesis: two.sided
##dispersion = 0.63, p < 0.0001 (not good)
plotResiduals(sim_res, clean_food_part$Sex_Age)

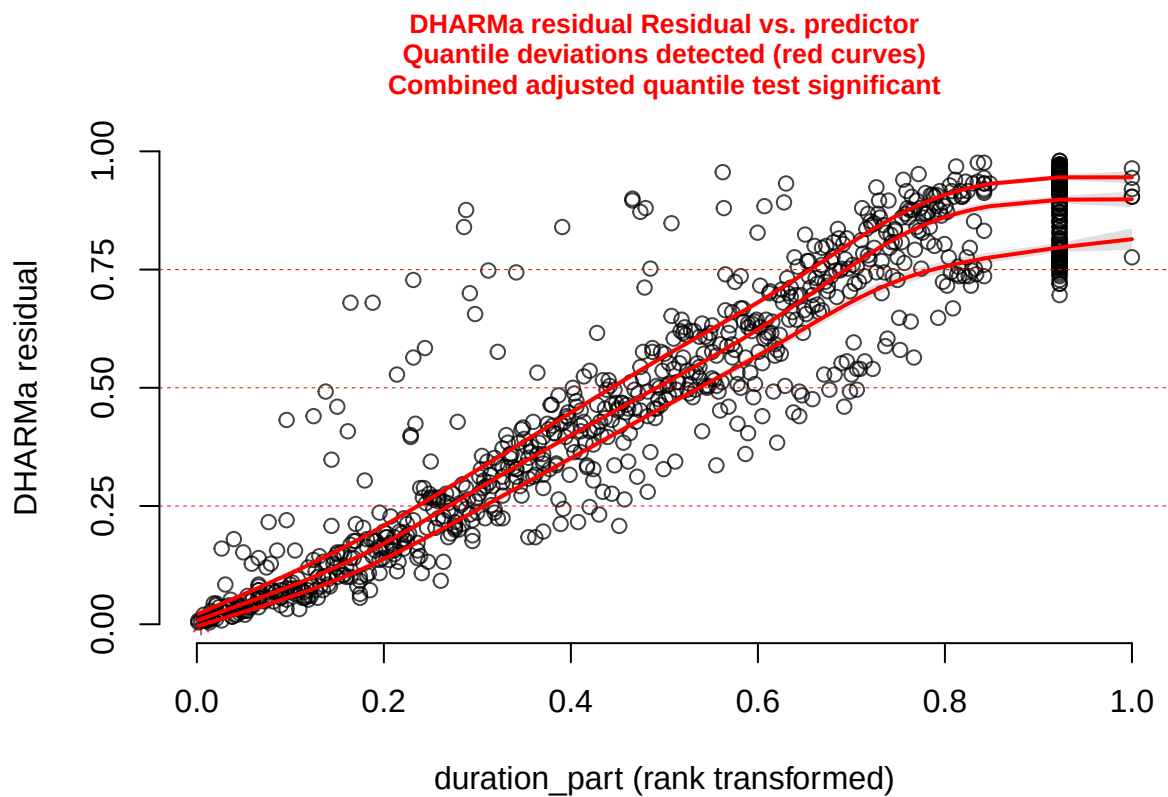
## Warning in ensurePredictor(simulationOutput, form): DHARMA::ensurePredictor:
## character string was provided as predictor. DHARMA has converted to factor
## automatically. To remove this warning, please convert to factor before
## attempting to plot with DHARMA.
```



```
plotResiduals(sim_res, clean_food_part$Food_part_clean)
```



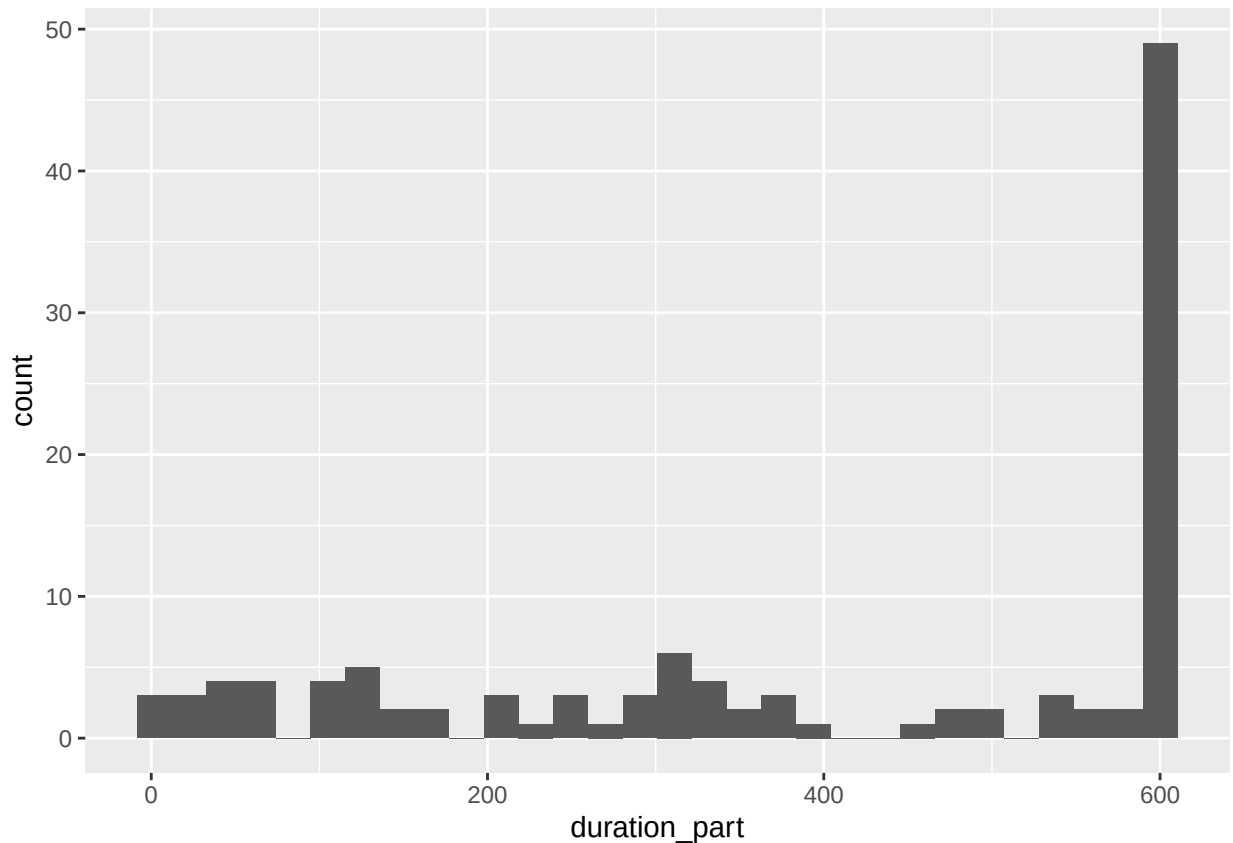
```
plotResiduals(sim_res, clean_food_part$duration_part)
```



*#so fruit is showing high variance, which may be driving this dispersion
#issue?
#is this enough of a fail to cancel the whole duration situation? or using a GLMM?*

```
#####fruit issue#####
ggplot(subset(clean_food_part, Food_part_clean == "fruit"),
  aes(x = duration_part)) +
  geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



*#so fruit actually has a lot of bouts that are 600seconds long, back to my orig
#question if gamma is correct...*

```
fruit_ceiling = clean_food_part %>%
  filter(Food_part_clean == "fruit") %>%
  summarise(
    total_bouts = n(),
    ceiling_bouts = sum(Duration_sec == 600, na.rm = TRUE),
    percent_ceiling = ceiling_bouts / total_bouts * 100
  )

table(clean_food_part$Sex_Age, clean_food_part$duration_part == 600)
```

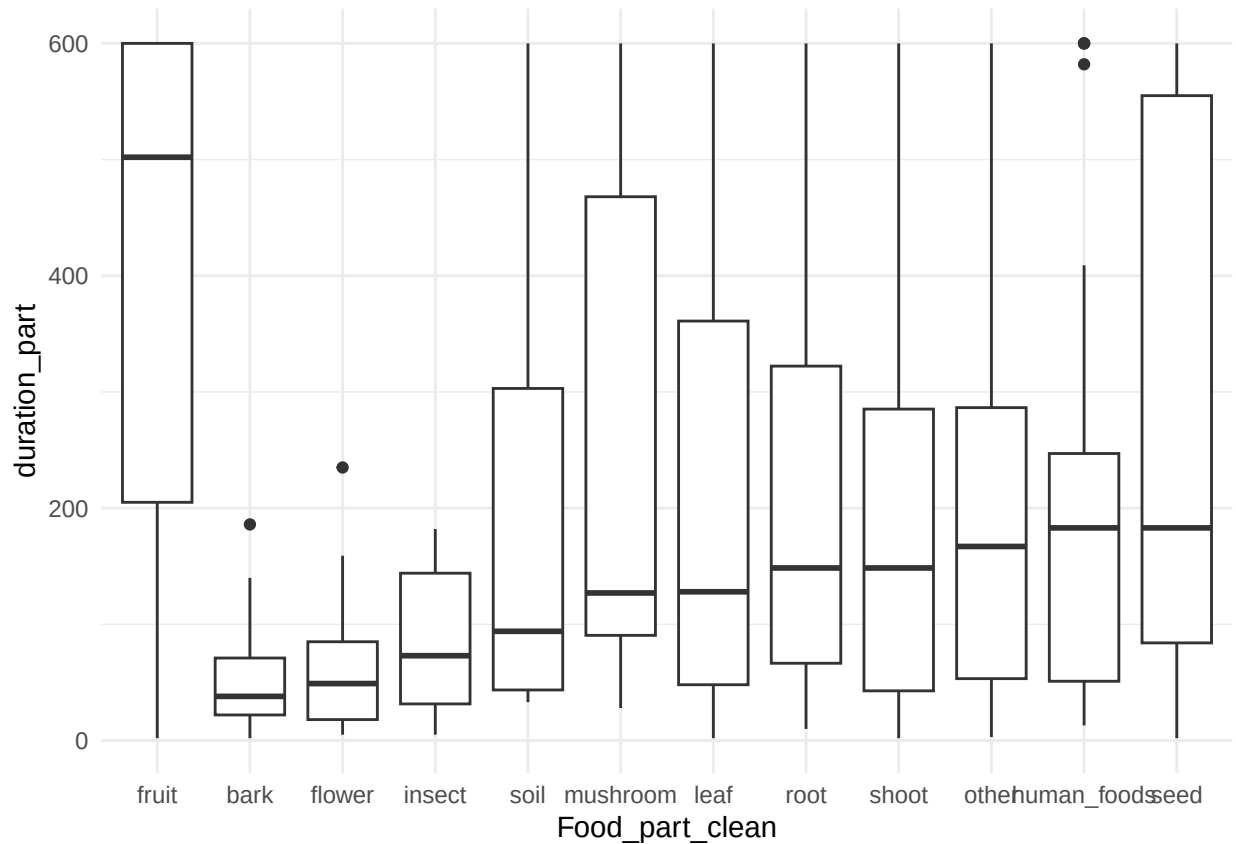
```
##
##           FALSE TRUE
## Adult female  486   75
## Juvenile     271   56
```

```
prop.table(table(clean_food_part$Sex_Age, clean_food_part$duration_part == 600), 1)
```

```
##
##           FALSE      TRUE
## Adult female 0.8663102 0.1336898
## Juvenile    0.8287462 0.1712538
```

#similar proportion of AF and J reaching 600 second ceiling

```
ggplot(clean_food_part, aes(x = Food_part_clean, y = duration_part)) +
  geom_boxplot() +
  theme_minimal()
```



```
clean_food_part$ceiling = clean_food_part$duration_part == 600
table(clean_food_part$Food_part_clean, clean_food_part$ceiling)
```

```
##
##      FALSE TRUE
## fruit      71  44
## bark       21   0
## flower     13   0
## insect      7   0
## soil        6   2
## mushroom    6   1
## leaf      194  27
## root        69   7
## shoot      228  20
## other       29   1
## human_foods 18   3
## seed       95  26
```

```
prop.table(table(clean_food_part$Food_part_clean, clean_food_part$ceiling), margin = 1)
```

```
##
##      FALSE      TRUE
```



```
## fruit      0.61739130 0.38260870
## bark      1.00000000 0.00000000
## flower    1.00000000 0.00000000
## insect    1.00000000 0.00000000
## soil      0.75000000 0.25000000
## mushroom  0.85714286 0.14285714
## leaf      0.87782805 0.12217195
## root      0.90789474 0.09210526
## shoot     0.91935484 0.08064516
## other     0.96666667 0.03333333
## human_foods 0.85714286 0.14285714
## seed      0.78512397 0.21487603
```

```
ggplot(clean_food_part, aes(x = Food_part_clean, fill = ceiling)) +
  geom_bar(position = "fill") +
  ylab("Proportion at 600s ceiling") +
  theme_minimal()
```

